Private Query Difference Explanations

KEYWORDS

Query refinement, diversity, provenance, ranking, top-k

PVLDB Reference Format:
. Private Query Difference Explanations. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:
The source code, data, and/or other artifacts have been made available at URL_TO_YOUR_ARTIFACTS.

1 INTRODUCTION

Modern data-management systems increasingly modify query results in order to satisfy external requirements such as fairness, representation, diversity, consistency, and data-quality constraints. Examples include fairness-aware query refinement systems that modify query predicates to satisfy demographic constraints [42, 49], provenance-based frameworks for bias analysis and mitigation [41], and consistent query answering (CQA) systems that repair inconsistent databases before evaluating queries [4, 10, 11]. Although these approaches successfully enforce the desired constraints, they often introduce substantial semantic changes whose consequences are difficult for users to understand.

To illustrate the problem, consider the following aggregate query over the Adult dataset. The query computes the average capital gain among highly educated individuals who work long hours. **(AG) Revise queries to have AVG or SUM, add example tuples for the dataset, highlight the changes in the query, and place in an example enviroment. (Alon) Addressed: income- based aggregation, example tuples and highlighting the changes in the query**

Example 1.1 (Query refinement over the Adult dataset). Consider the following simplified records from the Adult dataset:

id	gender	education	occupation	hoursPerWeek	income
1	Male	Bachelors	Exec-managerial	60	1
2	Male	Bachelors	Prof-specialty	55	1
3	Female	Masters	Prof-specialty	40	0
4	Female	Masters	Adm-clerical	35	0
5	Male	Masters	Exec-managerial	52	1

Table 1: Simplified Adult records. The binary attribute `income` equals 1 for individuals earning more than 50K and 0 otherwise.

The original query computes the fraction of high-income individuals among highly educated employees who work long hours:

```
SELECT AVG(income) AS highIncomeRate
FROM Adult
WHERE education IN ('Bachelors','Masters')
  AND hoursPerWeek >= 50;
```

Suppose that the population selected by this query violates a representation constraint requiring at least 25% female records. A query-refinement mechanism may therefore produce the following refined query, where the modified predicate values are shown in bold:

```
SELECT AVG(income) AS highIncomeRate
FROM Adult
WHERE education = 'Masters'
  AND hoursPerWeek >= 30;
```

The refined query changes the population over which the aggregate is computed. In the simplified records above, ids 1 and 2 may be removed because their education level is Bachelors, while ids 3 and 4 may be added because the hours-per-week threshold was relaxed. Thus, the refinement does not merely change the aggregate value; it changes the underlying population whose income distribution is being summarized.

For example, the refinement may increase female representation from 19.5% to 30.6%, thereby satisfying the representation constraint. However, understanding why the refinement succeeds is more difficult than verifying that the constraint is satisfied. Did the refinement simply introduce more women into the result? Did it alter the occupational composition of the selected population? Did it change the income distribution? Did it affect particular demographic groups?

As we show later, the refinement succeeds not merely because it increases female representation, but because it replaces a population dominated by high-income workers with intensive work schedules by a substantially different population with distinct occupational and economic characteristics. Understanding and communicating such semantic changes is the central goal of this work.

Although existing refinement systems provide the modified query, they offer limited support for explaining the semantic consequences of the refinement [42, 49].

The same challenge arises beyond query refinement. In consistent query answering, the answer produced from a repaired database may differ substantially from the answer obtained from the original inconsistent database [4, 10, 11]. Similarly, data-cleaning pipelines, provenance-based bias-mitigation frameworks, and fairness-aware analytics frequently generate multiple related query outputs whose semantic differences must be understood and justified [41, 50].

A natural solution is to provide explanations describing the semantic differences between two query outputs. However, generating such explanations is challenging.

First, multiple semantic factors may simultaneously contribute to the observed difference between two query outputs. Changes in demographic composition, education levels, occupations, income distributions, or family characteristics may all contribute to the

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit https://creativecommons.org/licenses/by-nc-nd/4.0/ to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

observed transformation. Identifying the most informative factors is therefore non-trivial.

Second, explanations must characterize the *difference* between two outputs rather than the properties of either output in isolation. Many characteristics may be common to both outputs and therefore irrelevant to understanding the observed change.

Third, explanations are often derived from sensitive demographic, financial, medical, or employment data. Releasing exact explanation statistics may inadvertently reveal information about specific individuals. Consequently, explanation generation must provide meaningful insight while satisfying strong privacy guarantees [29].

Existing work studied query refinement under diversity constraints [42, 49], fairness-aware data processing and bias mitigation [7, 41, 50], and database repairs and consistent query answering [4, 10, 11].

The database community has also developed several approaches for explaining query results and data transformations. Examples include provenance-based explanations, interactive explanation systems such as FEDEX [22], Tableau Explain Data [21], and recent differentially private explanation frameworks such as DPXPlain [63].

However, these approaches focus primarily on explaining a single query result, a prediction, or a data transformation. They do not address the problem of explaining the semantic differences between two related query outputs, such as an original query and its refined version or a query evaluated before and after a database repair.

Furthermore, existing explanation systems typically assume direct access to the underlying statistics or focus on releasing local explanations. To the best of our knowledge, no prior work provides a framework for generating *differentially private explanations of semantic differences between related query outputs*.

In this paper, we address this gap by introducing a framework for generating differentially private explanations of query-output differences. Rather than explaining a query result in isolation, we focus on explaining the semantic changes between two related outputs, such as an original query and its refined version or a query evaluated before and after a database repair.

Our framework identifies informative explanations through a two-stage process.

First, it privately selects promising semantic regions of the data according to two complementary criteria:

- **Representativeness**, which favors regions supported by statistically meaningful portions of the underlying dataset.
- **Coverage**, which measures the amount of query-output evidence contained in a region and favors regions supported by substantial numbers of tuples from the compared query outputs.

The framework then privately selects explanations from the chosen regions according to two additional criteria:

- **Interestingness**, which measures the extent to which an attribute distribution differs between the two query outputs.
- **Diversity**, which encourages the selected explanations to highlight different attributes and therefore provide multiple perspectives on the observed query transformation.

The framework first privately selects promising semantic regions of the data, then privately chooses explanation candidates using their interestingness and diversity contribution, and finally releases noisy histogram summaries using standard differential privacy mechanisms [29].

The resulting explanations provide meaningful insight into the effects of query refinements and repairs while preserving individual privacy.

Although motivated by fairness-aware query refinement, the proposed framework naturally generalizes to a broad range of data-management tasks, including data cleaning, provenance analysis, query debugging, and consistent query answering. Our main contributions are as follows:

- We introduce the problem of generating differentially private explanations for semantic differences between related query outputs.
- We propose a view-based explanation model and a utility framework that combines representativeness, coverage, interestingness, and diversity.
- We develop a differentially private explanation-generation algorithm that combines private view selection, private explanation selection, and noisy histogram release.
- We provide formal privacy guarantees for the complete explanation pipeline.
- We experimentally evaluate the framework on multiple real-world datasets and demonstrate that it produces informative and interpretable explanations while preserving privacy.

2 PRELIMINARIES

In this work, we study the problem of generating differentially private explanations for semantic differences between query outputs. This section introduces the formal concepts and notation used throughout the paper.

2.1 Databases, Queries, and Predicate Views

Databases. Let D be a relational database over schema $\mathcal{R}$ [1]. Each tuple $t \in D$ corresponds to an individual entity or record, and we write $N = |D|$ for the number of tuples. We denote by $Attr(D) = \{A_1, \ldots, A_m\}$ the set of attributes appearing in the schema of D. Each attribute A has a domain $\mathrm{Dom}(A)$, which may be categorical or numerical, and each tuple $t \in D$ assigns a value $t[A] \in \mathrm{Dom}(A)$ to every attribute A. Throughout, $\mathrm{Dom}(A)$ is the domain declared by the schema and is therefore *public*, since it does not depend on the contents of D. This distinction matters once privacy enters the picture, since a domain read off the data at hand would itself disclose information about the data (Section 2.5).

Queries. We consider SQL-style selection and projection queries together with aggregate queries, all of them having Boolean selection predicates over a single relational table D. A query Q consists of a selection predicate φ over $Attr(D)$, and aggregate queries have the form

$$\texttt{SELECT } agg(A) \texttt{ FROM } D \texttt{ WHERE } \varphi,$$

where $agg \in \{\texttt{COUNT}, \texttt{SUM}, \texttt{AVG}\}$ and φ is a Boolean combination of *atomic conditions*, namely numerical comparisons such as $A \le c$ or $A \ge c$, categorical equality predicates such as $A = v$, and categorical set-membership predicates such as $A \in S$.

Our explanations describe the *population* a query selects rather than the scalar it may return, since it is the population whose composition changes when a query is refined or a database is repaired, and an aggregate value alone admits no distributional summary. We therefore write

$$Q(D) \;=\; \sigma_{\varphi}(D) \;=\; \{t \in D \mid \varphi(t) = \texttt{true}\}$$

for the *output* of Q on D, that is, the set of tuples selected by Q. For an aggregate query the reported value is a function of $Q(D)$, and all statistics defined below are taken over $Q(D)$.

Predicate views. To explain semantic differences between query outputs, our framework analyzes subsets of the data defined by predicates. Predicate-defined subpopulations are commonly used in explanation and query-analysis frameworks to localize meaningful changes in the data [22, 63, 67].

Definition 2.1 (Predicate View). Given a database D and a predicate p, the predicate view induced by p is

$$p(D) = \{t \in D \mid p(t) = \texttt{true}\}.$$

The framework is given a finite collection $\mathcal{P}$ of view predicates, which localize the explanations to specific semantic regions of the data. We take $\mathcal{P}$ to consist of conjunctions of at most ℓ atomic conditions over a designated set of view attributes, with ℓ small (in our experiments $\ell \le 2$); its members are enumerated from the public schema and domain metadata, so both $\mathcal{P}$ and its size $|\mathcal{P}|$ are fixed independently of the contents of D. Two further public attribute sets are used. The *explanation attributes* $Attrs \subseteq Attr(D)$ are those whose distributions may be reported, and the *reference attributes* $\mathcal{A}_R \subseteq Attr(D)$ are those against which a view is compared to the full database in Section 3.2.

Evaluating a query inside a view is the basic operation of the framework, and the following identity is what makes it well behaved.

Proposition 2.2 (View restriction). *For every query Q with selection predicate φ and every view predicate p,*

$$Q(p(D)) \;=\; \sigma_{\varphi \wedge p}(D) \;=\; Q(D) \cap p(D).$$

In particular $Q(p(D)) \subseteq p(D)$ and $Q(p(D)) \subseteq Q(D)$.

Proof. Both Q and p act on the same single relation by selection, and selections compose, so that $t \in \sigma_{\varphi}(\sigma_p(D))$ holds iff $t \in D$, $p(t)$ and $\varphi(t)$, which is the definition of $\sigma_{\varphi \wedge p}(D)$, and also of $\sigma_{\varphi}(D) \cap \sigma_p(D)$. The two containments are immediate. □

The containment $Q(p(D)) \subseteq p(D)$ is used repeatedly. It is why the histogram of a query output inside a view is supported on the bins of the view itself (Definition 3.4), and why coverage inside a view is bounded by the view size (Section 3.2).

Throughout the paper we study pairs of related queries (Q_1, Q_2) evaluated over the same database D, and our goal is to explain the semantic differences between the outputs $Q_1(D)$ and $Q_2(D)$. The next subsection describes two application settings that naturally give rise to such pairs.

Example 2.3 (Running example). We use the Adult refinement of Section 1 as a running example. The database D holds census records over attributes including `gender`, `education`, `occupation`, `hoursPerWeek` and `income`. The original query Q_1 and its refinement Q_2 have selection predicates

$$\varphi_1 \;=\; \texttt{education} \in \{\texttt{Bachelors}, \texttt{Masters}\} \;\wedge\; \texttt{hoursPerWeek} \ge 50,$$

$$\varphi_2 \;=\; \texttt{education} = \texttt{Masters} \;\wedge\; \texttt{hoursPerWeek} \ge 30.$$

so that $Q_1(D)$ and $Q_2(D)$ are the two populations to be compared. A view predicate such as $p = (\texttt{education} = \texttt{Masters})$ isolates a semantic region of D, and by Proposition 2.2, $Q_1(p(D))$ is exactly the set of Master's-degree holders working at least 50 hours per week.

2.2 Query Refinement and Consistent Query Answering

Both settings we consider derive a second output from an original query, and it is the resulting pair that our framework takes as input. We state them formally.

Query refinement. Refinement modifies a query so that its output meets a set of constraints on the composition of the result while staying close to the original query [15, 41, 42, 49].

Definition 2.4 (Output constraint). Let $G \subseteq Attr(D)$ be a set of grouping attributes and let g be an assignment of values to G, identifying a group of tuples. A *cardinality constraint* is a triple $\langle g, \alpha, \beta \rangle$, satisfied by a set of tuples T when

$$\alpha \;\le\; |\{t \in T \mid t[G] = g\}| \;\le\; \beta,$$

and a *proportional constraint* replaces this count by its fraction of $|T|$. For a set C of such constraints we write $T \models C$ when T satisfies every member of C.

Definition 2.5 (Query refinement). Let Q have selection predicate φ, and let Φ_{φ} be the space of predicates obtained from φ by relaxing, tightening, removing or replacing its atomic conditions. Given D, a constraint set C and a deviation measure d on predicates, a *refinement* of Q is a query Q' with predicate $\varphi' \in \Phi_{\varphi}$ such that $Q'(D) \models C$. A refinement mechanism returns such a Q', typically one minimizing $d(\varphi, \varphi')$.

Consistent query answering. CQA evaluates a query over a database that may violate a set of integrity constraints, by reference to the instances that repair those violations [4, 10, 11].

Definition 2.6 (Repair). Let Σ be a set of integrity constraints. An instance D' is a *repair* of D with respect to Σ if $D' \subseteq D$, $D' \models \Sigma$, and no proper superset of D' contained in D satisfies Σ. We write $\mathrm{Repairs}(D, \Sigma)$ for the set of all repairs of D.

Definition 2.7 (Consistent answer). The *consistent answer* to a query Q over D with respect to Σ is

$$Q_{\Sigma}(D) = \bigcap_{D' \in \mathrm{Repairs}(D,\Sigma)} Q(D'),$$

the set of tuples returned by Q on every repair of D.

We take repairs to be subsets of D, as in Definition 2.6, so that $Q_{\Sigma}(D) \subseteq Q(D) \subseteq D$ and the consistent answer is again a subpopulation of D that the histograms of Section 2.3 can summarize.

The induced pair. Each of the two settings supplies the framework with a pair of outputs over the same database,

$$(Q(D), Q'(D)) \qquad \text{and} \qquad (Q(D), Q_\Sigma(D)),$$

for refinement and for CQA respectively. The framework requires only that each element of the pair is a subset of D, and it takes the restriction of an output $T \subseteq D$ to a view p to be $T \cap p(D)$, which coincides with $Q(p(D))$ by Proposition 2.2 whenever $T = Q(D)$ for a selection query Q. Our goal is not to compute refinements, repairs or consistent answers. The mechanism producing the second element is treated as a black box, and the task is to explain the difference between the two elements while preserving privacy.

2.3 Histograms and Distributions

Our explanations are based on distributions of attributes over query outputs and predicate views. Both are sets of tuples, so we define the basic statistics for an arbitrary finite set T of tuples over $Attr(D)$; the cases of interest are $T = D$, $T = p(D)$, and $T = Q_i(p(D))$.

Definition 2.8 (Histogram). Given a finite set of tuples T and an attribute $A \in Attr(D)$, the histogram of A over T is the function

$$H_{T,A} : \mathrm{Dom}(A) \to \mathbb{N}, \qquad H_{T,A}(v) = |\{t \in T \mid t[A] = v\}|,$$

so that $H_{T,A}(v)$ is the number of tuples in T whose value for A equals v.

Definition 2.9 (Empirical distribution). For a non-empty finite set of tuples T and an attribute A, the empirical distribution of A over T is

$$P_{T,A}(v) = \frac{H_{T,A}(v)}{|T|}, \qquad v \in \mathrm{Dom}(A),$$

equivalently $P_{T,A}(v) = \Pr(t[A] = v \mid t \in T)$ for t drawn uniformly from T. It is undefined for $T = \emptyset$; the convention for an empty view is fixed in Section 3.2.

In the standard statistical sense, a marginal distribution is obtained by projecting a joint distribution onto a subset of attributes. In this paper, we use one-way marginals, namely marginals over a single attribute. For a query output $Q(D)$ and an attribute A, the one-way marginal of A is $P_{Q(D),A}$. These one-way marginals are the basic objects used in our explanations, since comparing $H_{Q_1(D),A}$ and $H_{Q_2(D),A}$, or their normalized counterparts, summarizes how the two query outputs differ with respect to attribute A. Such distributional summaries are also widely used in privacy-preserving data analysis and release mechanisms [29].

For ease of presentation, we assume that the attribute values used in histograms and predicate views are categorical. Numerical attributes are incorporated by rendering them categorical, e.g., by bucketizing their domains into ranges, a common practice when presenting aggregate results. Thus, for categorical attributes the histogram bins are the attribute values, while for numerical attributes the bins correspond to the chosen ranges. In either case the bins are fixed in advance from the public schema and domain metadata and never from the data at hand, so that the set of bins carries no information about D; we write $\mathrm{Dom}(D[A])$ for this public set of bins of attribute A and use it as the index set of every histogram over A.

(Alon) Done: formal Bucketing definition removed and replaced with an explanation and motivation for using buckets for numerical attributes.

(Alon) Done: unified with the statistics discussion above.

To quantify semantic differences between query outputs, we use statistical distance measures between distributions.

Definition 2.10 (Total Variation Distance). Given two distributions P_1 and P_2 over the same domain Ω, the Total Variation Distance (TVD) between them is

$$TVD(P_1, P_2) = \frac{1}{2} \sum_{x \in \Omega} |P_1(x) - P_2(x)| \ \in [0, 1].$$

TVD is commonly used to measure similarity between distributions and is used throughout our framework for view filtering and representativeness analysis. We reserve the symbols Q, Q_1 and Q_2 for queries and use P_1, P_2 for generic distributions.

2.4 Explanation Quality Measures

The criteria our framework optimizes, namely *interestingness*, *coverage*, *representativeness* and *diversity*, are not new to this paper. Each has an established meaning in data mining, data exploration and query explanation. Our contribution is to instantiate them for a *pair* of query outputs over the same database, and to replace them by low-sensitivity counterparts that can be optimized under differential privacy (Section 4). We review the classical forms here; the specific definitions used in this paper appear in Section 3.2.

Interestingness. A pattern is interesting when it deviates from what a reference model would lead one to expect; this is the *unexpectedness* view of interestingness introduced by Silberschatz and Tuzhilin [61] and surveyed, together with the many objective measures instantiating it, by Geng and Hamilton [33] and by Tan et al. [62]. In data management the same idea appears as deviation from an expected value in the discovery-driven exploration of OLAP cubes [59], and as deviation between distributions in visualization recommendation, where SeeDB scores a candidate view by the distance between the distribution it induces on a target subset and the distribution it induces on a reference subset [65], and FEDEX scores an exploration step by how much it changes the distribution of an attribute [22]. A complementary line measures the *influence* of tuples or predicates on an aggregate answer rather than the movement of a distribution [58, 67]. Our interestingness (Section 3.2) is of the distributional kind, with the two compared populations being the two query outputs $Q_1(p(D))$ and $Q_2(p(D))$ rather than a subset and its complement. It compares their histograms bin by bin, normalizing each bin's difference by the size of that bin in the region, so that it remains defined, and measures a difference, even when only one of the outputs meets the region.

Coverage. A deviation observed on a handful of tuples is not evidence, and every explanation framework therefore pairs a deviation measure with a mass measure. The oldest form is the *support* of association-rule mining [3]; the same role is played by the support threshold that MacroBase applies to its risk-ratio explanations [6], by the support predicate of the Diff operator [2], by the *coverage* of an anchor rule in model explanation [57], and by the size budget of explanation tables [32]. Coverage is also studied as a first-class

property of a dataset in fairness-aware data preparation [5]. Our coverage differs from all of these in counting the evidence of *both* outputs jointly. Since the object being explained is a comparison, a region is useful when the two outputs together place enough tuples in it, including when only one of them does: a region that one output fills and the other leaves empty is where a refinement or repair adds or removes part of the population wholesale, the largest difference a region can show. This is what motivates the form of coverage in Section 3.2.

Representativeness. Subgroup discovery makes explicit the tension between how unusual a subgroup is and how general it is, and resolves it with quality functions that multiply the two, the Klösgen family $n^a(p - p_0)$ and weighted relative accuracy being the canonical examples [36, 66]. Our representativeness plays the same role with the direction of the requirement reversed. Because the deviation we report is the *change* inside a region, the region itself should be typical of the database rather than exceptional, so a view is scored by the closeness of its attribute distributions to the global ones. This guards against explanations that are dramatic only because they describe an idiosyncratic corner of the data.

Diversity. When several results are returned together, their marginal value depends on what has already been shown; maximal marginal relevance [16] is the classical formulation and the axiomatic treatment of Gollapudi and Sharma [35] and the survey of Drosou and Pitoura [25] develop it for query and search results. The same principle is applied to sets of explanations in data exploration [22, 51]. Diversity objectives of this shape are typically submodular, and are optimized greedily [44]; the resulting guarantee is recalled below.

Set-level selection. Because our framework releases k explanations rather than one, its objective is a set function. Diversity objectives of this kind are usually optimised greedily and defended by the $(1 - 1/e)$ guarantee that monotone submodularity buys [30, 53]. The objective of Section 3.2 admits a stronger statement, established in Appendix 7: greedy returns its exact maximiser.

Quality measures under differential privacy. None of the measures above is designed with privacy in mind, and each is a ratio or a product of counts whose value can be moved substantially by a single tuple. Making them usable inside a private selection mechanism is therefore a design problem in its own right, addressed for aggregate-query explanations by DPXPlain [63] and for cluster explanations by DPClustX [34], both of which replace their natural quality measures by low-sensitivity surrogates. Section 3.3 quantifies the difficulty in our setting and Section 3.2 constructs the measures we use.

2.5 Differential Privacy

Because explanations are derived from sensitive data, our framework relies on differential privacy to ensure that released explanations reveal little about any particular individual [29]. A differentially private mechanism behaves almost the same on two databases that differ in one tuple, so whether a single individual is included in the database has only a limited effect on the distribution of the released output. The parameter ϵ controls the privacy loss, and smaller values give stronger privacy at the cost of more noise.

Definition 2.11 (Neighboring Datasets [28]). Two datasets D, D' are neighboring, written $D \sim D'$, if one is obtained from the other by adding or removing a single tuple, that is, if $D' = D \cup \{t\}$ or $D' = D \setminus \{t\}$ for some t.

We adopt this *unbounded* form of the relation throughout, and all sensitivity statements in Sections 3 and 4 are with respect to it.

Definition 2.12 (Differential Privacy [28, 29]). A randomized mechanism $\mathcal{M}$ satisfies ϵ-differential privacy if for every pair of neighboring datasets $D \sim D'$ and every subset of outputs S,

$$Pr[\mathcal{M}(D) \in S] \leq \exp(\epsilon) Pr[\mathcal{M}(D') \in S].$$

The noise a mechanism must add is calibrated to the sensitivity of the quantity it releases.

Definition 2.13 (Global Sensitivity [29]). The global sensitivity of a function $f : \mathcal{D} \to \mathbb{R}^d$ is

$$\Delta f = \max_{D \sim D'} \|f(D) - f(D')\|_1.$$

Definition 2.14 (Laplace Mechanism [28, 29]). Given a function $f : \mathcal{D} \to \mathbb{R}^d$ with global sensitivity Δf, the Laplace mechanism releases

$$f(D) + (Y_1, \ldots, Y_d), \qquad Y_1, \ldots, Y_d \overset{\text{i.i.d.}}{\sim} \text{Laplace}(\Delta f/\epsilon),$$

where Laplace(b) denotes the zero-mean Laplace distribution with scale b, whose density is $\frac{1}{2b} \exp(-|x|/b)$.

Thus, functions with lower sensitivity require less noise for the same privacy budget, which motivates the low-sensitivity form of the measures used in our algorithm. A quality score written as a ratio of counts is the awkward case, because its sensitivity is dictated by the possibility of a vanishing denominator rather than by the candidates the mechanism actually meets. One response is to calibrate the noise to the database at hand, as smooth sensitivity [55] and propose-test-release [27] do; both buy their tighter noise with a relaxation such as (ϵ, δ)-DP. Section 3.2 takes the other route, raising every denominator by a constant fixed in advance and independent of the data, which keeps the guarantee in pure ϵ-DP at the cost of protecting against a worst case that may never be realised. When the released object is a histogram, d is the number of bins and the noise is drawn independently per bin.

Definition 2.15 (Exponential Mechanism [29, 46]). Given a dataset D, a set of candidate outputs Y, a utility function $u : Y \times D \to \mathbb{R}$, and privacy budget ϵ, the exponential mechanism selects an output $y \in Y$ with probability proportional to

$$\exp\left(\frac{\epsilon u(y, D)}{2\Delta u}\right),$$

where Δu is the global sensitivity of the utility function.

The exponential mechanism enables private selection of high-quality explanation candidates while preserving differential privacy.

Our algorithms select not one candidate but the top k, which the following mechanism does in a single pass.

Definition 2.16 (One-Shot Top-k Mechanism [26]). Given a dataset D, candidates Y, a utility function u with global sensitivity Δu, a budget ϵ and an integer k, the mechanism perturbs every candidate score once,

$$\tilde{u}(y) = u(y, D) + \text{Gumbel}\left(\frac{2k\,\Delta u}{\epsilon}\right),$$

where Gumbel(b) is the zero-location Gumbel distribution with scale b and the draws are independent across candidates, and returns the k candidates with the largest $\tilde{u}$, in that order.

Theorem 2.17 ([26]). *The one-shot top-k mechanism satisfies ϵ-differential privacy.*

The guarantee follows because the mechanism is distributionally identical to applying the exponential mechanism k times without replacement, each with budget ϵ/k, so sequential composition (Theorem 2.18) applies; the practical gain is that the scores are perturbed once rather than recomputed in each of the k rounds. The same mechanism is used for private top-k selection by DPXPlain [63] and by the clustering-explanation framework of [34].

Differential privacy satisfies several important composition properties, enabling complex privacy-preserving analytical workflows constructed from multiple private mechanisms [13, 43, 47]. These properties are central to the privacy analysis of our algorithms, which combine private view selection, private candidate selection, and noisy histogram release.

Theorem 2.18 (Sequential Composition [29, 43]). *Let $\mathcal{M}_1, \ldots, \mathcal{M}_k$ be randomized mechanisms such that $\mathcal{M}_i$ satisfies ϵ_i-differential privacy for every $i \in \{1, \ldots, k\}$. Then the mechanism that releases the joint output*

$$\mathcal{M}(D) = (\mathcal{M}_1(D), \ldots, \mathcal{M}_k(D))$$

satisfies

$$\left(\sum_{i=1}^{k} \epsilon_i\right)\text{-differential privacy.}$$

The statement holds in the adaptive *setting as well. If the choice of $\mathcal{M}_i$ may depend on the outputs of $\mathcal{M}_1, \ldots, \mathcal{M}_{i-1}$, the same bound $\sum_i \epsilon_i$ applies.*

We rely on the adaptive form, since the utility optimised in each round of Algorithm 2 depends on the explanations released in earlier rounds.

Sequential composition allows us to account for the total privacy loss of an algorithm that invokes multiple differentially private subroutines by summing their individual privacy budgets.

Theorem 2.19 (Post-Processing [29, 43]). *Let $\mathcal{M}$ be an ϵ-differentially private mechanism, and let f be any randomized mapping that does not access the private database except through the output of $\mathcal{M}$. Then the mechanism*

$$f(\mathcal{M}(D))$$

also satisfies ϵ-differential privacy.

The post-processing property implies that any deterministic or randomized computation performed only on already-private outputs does not incur additional privacy loss. In particular, the constants τ and γ of Equations (13) and (14), being functions of public quantities alone, consume no privacy budget.

3 PROBLEM STATEMENT AND MODEL

This section states what an explanation is and what we are looking for (Section 3.1), gives the measures of explanation quality as modifications of the forms used in prior work (Section 3.2), bounds the sensitivity of the smoothed histogram distance and shows that the forms used in prior work admit none that decays with the database (Section 3.3), and derives from that the two-stage structure the framework uses and the two utilities it optimises (Section 3.4). **(AG) Slightly overstated. Section 3.3 bounds the smoothed Histogram Distance and gives lower bounds for the three forms used in prior work, but it gives no bound for Rep_{γ_R} or *Cov* on its own, and the zero sensitivity of *Div* is stated where *Div* is defined. Suggest rephrasing to what it actually does, and likewise for the closing sentence of the Section 3.2 preamble.**

3.1 Private Explanations of Query Differences

We consider a trusted server that holds a private database D and two related queries Q_1 and Q_2 over it. The second query is typically derived from the first by a mechanism that the framework treats as a black box. In query refinement (Definition 2.5), $Q_1 = Q$ is the original query and $Q_2 = Q'$ its refinement; in consistent query answering (Definition 2.7), Q_2 is Q_1 evaluated under repair semantics. Both are instances of the same problem, explaining how two related queries differ on the same database, and the framework does not depend on which one produced the pair. For a view p we write $Q_i(p(D))$ for the restriction of the output of Q_i to the view, $Q_i(D) \cap p(D)$ (Section 2.2); for a selection query it is the result of evaluating Q_i on $p(D)$ (Proposition 2.2).

Our goal is to let an analyst see *how* the two outputs differ, which parts of the population the change added, removed or reshaped, while revealing nothing about any individual in D. Two things have to be settled for that: what an explanation looks like, and what makes one good. This subsection settles the first, and Section 3.2 the second; given those quality measures, the task is to return a handful of explanations that score well under them, privately.

Workflow. The analyst states what is to be explained and under what budget (Figure 1): the two queries Q_1 and Q_2, the collection $\mathcal{P}$ of view predicates to search over, the explanation attributes *Attrs*, the number of explanations k and the privacy budget ϵ. Both queries are given. How Q_2 was obtained, whether from a refinement algorithm, a repair procedure or the analyst writing it by hand, happens before the framework runs and is not part of it; the same holds for $\mathcal{P}$, which the analyst either supplies or has generated from the schema under the bound of Section 2.1. Algorithm 1 takes one further argument, the diversity weight λ, which does not say what to explain but tunes how strongly the second stage spreads the explanations across attributes. The reference smoothing constant γ_R is not an argument either: Eq. (4) fixes it from N alone, and Proposition 3.11 holds at every public choice of it. The two constants that matter most, how many views the first stage keeps and how hard the second stage smooths, the algorithm fixes for itself in Section 4.1. The server then evaluates both queries, privately selects a small set of views from $\mathcal{P}$ and, within them, the explanation candidates that best characterise the difference, and releases noisy histograms for the selected candidates. Only this last object leaves

the server; the analyst never observes D, the exact outputs, or the exact scores that drove the selection. Because Q_2 arrives as an input, the privacy analysis accounts for the explanation alone; if the mechanism that produced Q_2 read D, its own privacy cost composes with ours and is outside our scope.

The structure of an explanation. An explanation answers *where* the two outputs differ and *in what respect*. The where is a region of the database, given by a view predicate $p \in \mathcal{P}$; the respect is an attribute $A \in \mathit{Attrs}$, whose distribution the two outputs are compared on.

Definition 3.1 (Histogram Explanation Candidate). Given a view $p \in \mathcal{P}$ and an explanation attribute $A \in \mathit{Attrs}$, the *histogram explanation candidate* $e = (p, A)$ induces the two histograms

$$H_{Q_1(p(D)),A} \qquad \text{and} \qquad H_{Q_2(p(D)),A}.$$

Writing $\mathrm{attr}(p)$ for the attributes the predicate p constrains, the space of candidates is

$$\mathcal{E} = \{(p, A) \in \mathcal{P} \times \mathit{Attrs} \ : \ A \notin \mathrm{attr}(p)\},$$

and we write $E \subseteq \mathcal{E}$ for a set of them, and A_c for the attribute of a candidate $c \in \mathcal{E}$. **(AG) C already denotes the output-constraint set in Definitions 2.4 and 2.5, both cited earlier in this section. Suggest renaming one of the two, for instance $\mathcal{E}$ for the candidate space here.** A view that fixes A leaves the two histograms supported on a single bin, so such a pair reports the change in one count rather than a change in a distribution; excluding it also keeps the candidate space aligned with the reference set of Section 3.2, from which $\mathrm{attr}(p)$ is excluded for the same reason.

Example 3.2 (Explanation candidates). Figure 2 shows two candidates for the refinement of Example 2.3, both on $A = \texttt{occupation}$. In panel (a) the view is $p = (\texttt{education} = \texttt{Masters})$: both queries meet it, and the refinement shifts the occupational mix towards professional-specialty roles rather than merely adding people. In panel (b) the view is $p' = (\texttt{education} = \texttt{Bachelors})$, which φ_2 cannot satisfy, so $Q_2(p'(D)) = \emptyset$ and the candidate reports a population the refinement removes in its entirety. Neither change is visible in the aggregate value, and the second is the kind of one-sided view the measures of Section 3.2 have to handle rather than discard.

What is released for a selected candidate is a perturbed version of its two histograms, in the sense of the Laplace mechanism (Definition 2.14).

Definition 3.3 (Noisy histogram). For a set of tuples $T \subseteq D$, an attribute A and a scale $b > 0$, the *noisy histogram* of A over T is

$$\hat{H}_{T,A}(v) = H_{T,A}(v) + Y_v, \qquad v \in \mathrm{Dom}(D[A]),$$

with Y_v drawn independently from the Laplace distribution of scale b, one draw per bin of the public domain of A.

We release the pair $\big(\hat{H}_{Q_1(p(D)),A}, \hat{H}_{Q_2(p(D)),A}\big)$ rather than only its difference, so that the analyst sees the two profiles themselves and not just the size of the gap between them. The pair costs more: a tuple selected by both queries changes both histograms, so the pair has L_1-sensitivity 2 against 1 for the difference (Proposition 4.1), which doubles the noise scale at a fixed budget. Where budget is scarce the difference can be released instead.

3.2 Measures of Explanation Quality

The criteria we optimise are the four reviewed in Section 2.4: interestingness, representativeness, coverage and diversity. None is new. What is new is the form we give them, and the reason for the change is always the same: a measure whose value can be swung by one individual cannot be optimised under differential privacy at all, because the noise a private mechanism must add is calibrated to exactly that swing. For the three criteria that read the database we therefore recall the form used in prior work, say what goes wrong with it here, and give the form we use; diversity reads only released outputs and needs no such change. Section 3.3 states the effect of all four changes in one place.

Interestingness. In its distributional form a candidate is scored by the distance between the two distributions it induces [22, 65], which in our setting is the total variation distance between the conditional distributions of A over $Q_1(p(D))$ and $Q_2(p(D))$, or twice it in the normalised form. Two things go wrong. It is undefined when one of the two outputs does not meet the view, which is exactly the case of the one-sided views of Example 3.2, where a refinement adds or removes a population wholesale. And because it is a ratio over the output sizes, a single tuple can move it by half of its entire range, however large the database (Proposition 3.10). We therefore compare the two outputs *bin by bin in counts*, normalised by the size of the bin in the view rather than by the size of the outputs, and add a public constant γ to each denominator.

Definition 3.4 (Smoothed Histogram Distance). Given a dataset D, a view predicate p, an attribute A, two queries Q_1, Q_2, and a public smoothing constant $\gamma \geq 0$,

$$HD_\gamma(D; p, A, Q_1, Q_2) = \sum_{v \in \mathrm{Dom}(D[A])} \frac{\big|H_{Q_1(p(D)),A}(v) - H_{Q_2(p(D)),A}(v)\big|}{H_{p(D),A}(v) + \gamma} \tag{1}$$

Setting $\gamma = 0$ recovers the unsmoothed score, in which case the sum must be restricted to bins with $H_{p(D),A}(v) > 0$ to avoid division by zero. For $\gamma > 0$ no such restriction is needed: a bin absent from $p(D)$ is necessarily absent from $Q_1(p(D))$ and $Q_2(p(D))$ as well, since both are subsets of $p(D)$, so it contributes exactly $0/\gamma = 0$.

The motivation for γ is a mismatch between the *worst case* the sensitivity bound must cover and the *typical case* the mechanism actually encounters. A bin containing two tuples can have its ratio swung by a full unit by a single individual, so the unsmoothed bound must accommodate that; yet the bound is then applied uniformly, including to candidates supported by thousands of tuples. The exponential mechanism consequently cannot separate candidates whose scores differ by far less than the bound. Smoothing raises every denominator by a fixed public amount, which barely perturbs well-supported bins while removing the pathological ones from consideration.

The resulting bound is strictly decreasing in γ, which is the entire point: γ buys a tighter worst case at the cost of a mild, uniform compression of every candidate's score. The same device appears in differentially private decision-tree induction, where split criteria built from ratios of counts are smoothed for this reason [31],

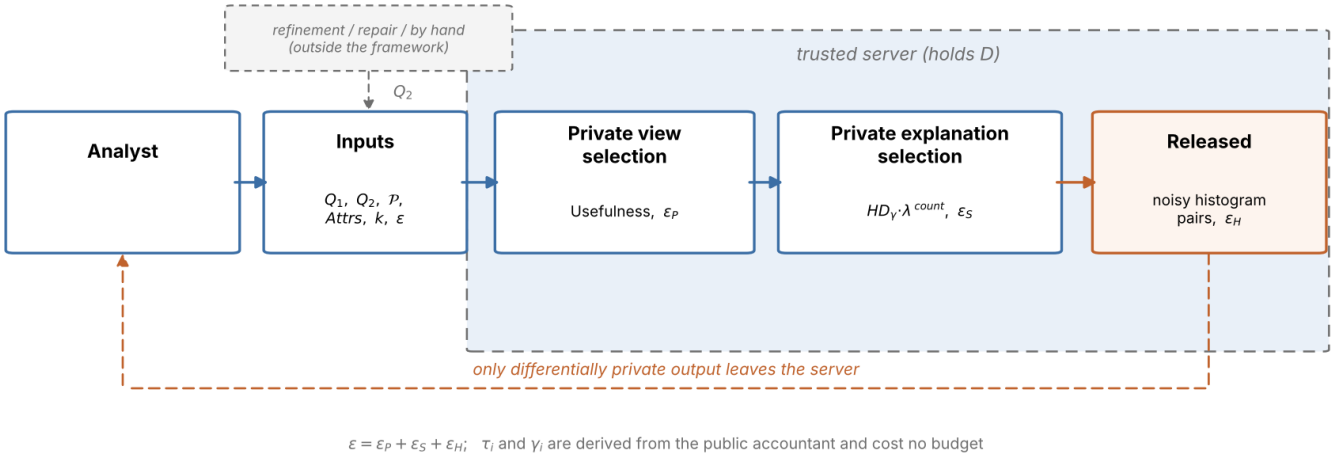


Figure 1: The private query-difference explanation framework. The analyst supplies both queries together with the search space and the budget; where Q_2 came from, whether a refinement algorithm, a repair procedure or the analyst, is outside the framework. The server privately selects views, then explanations, and releases only noisy histogram pairs. The constants τ and γ are fixed from public quantities and therefore consume no budget.

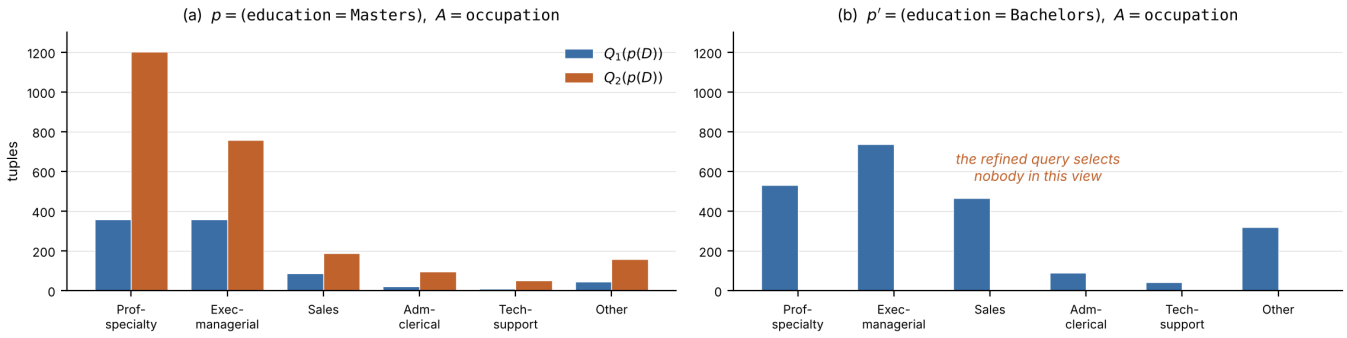


Figure 2: Two explanation candidates on Adult, for Q_1 selecting Bachelor's and Master's degree holders working at least 50 hours and Q_2 selecting Master's degree holders working at least 30 (Example 3.2). Counts are exact; what the framework releases is each pair of bars perturbed with Laplace noise.

and in [34], which rescales its cluster-quality measures by the cluster size; Section 2.5 contrasts it with calibrating noise to the data instead.

Representativeness. Subgroup discovery scores a region by trading its unusualness against its generality [36, 66]; we need the requirement in the opposite direction, since the deviation we report is the change *inside* a region and the region itself should be typical of the database. The natural form is $1 - TVD$ between the distribution of an attribute inside the view and in the whole database. Here too the view-side term d_v/m is a ratio whose denominator is the view size, so a single tuple can rewrite the profile of a near-empty view (Proposition 3.10), and the measure is undefined when the view is empty. That instability is a question of evidence rather than of privacy: Proposition 3.11 holds at every γ_R, the product with coverage absorbing the sensitivity, since $Cov \le 2m/N$ vanishes on exactly the views whose profile is unstable. But a profile built on a handful of tuples is not worth reading at face value, whatever its sensitivity. The same smoothing repairs both.

Definition 3.5 (Smoothed Representativeness). Let c_v^A and d_v^A be the number of tuples with A-value v in D and in $p(D)$, let $N = |D|$ and $m = |p(D)|$, and let $\gamma_R \ge 0$ be a public constant. For a single attribute,

$$Rep^A_{\gamma_R}(p, D) = 1 - \frac{1}{2} \sum_{v \in \mathrm{Dom}(D[A])} \left| \frac{c_v^A}{N} - \frac{d_v^A}{m + \gamma_R} \right|, \qquad (2)$$

and the representativeness of the view is the average over the reference attributes it does not itself constrain,

$$Rep_{\gamma_R}(p, D) = \frac{1}{|\mathcal{A}_R(p)|} \sum_{A \in \mathcal{A}_R(p)} Rep^A_{\gamma_R}(p, D), \qquad (3)$$

where $\mathcal{A}_R(p) = \mathcal{A}_R \setminus \mathrm{attr}(p)$.

Two conventions are fixed by Eq. (3). The reference set $\mathcal{A}_R$ is chosen in advance and does not depend on the candidate attribute, since representativeness is a property of the view and is evaluated before views are paired with attributes. And the attributes the view itself constrains are excluded, because comparing $p(D)$ with D on the attribute that defines p is close to tautological: a view `occupation` = c necessarily has an extreme occupation distribution. We assume $|\mathcal{A}_R| > \ell$, so that $\mathcal{A}_R(p)$ is never empty.

How large should γ_R be? Of the mass Eq. (2) weighs against D, the view supplies $m/(m+\gamma_R)$ and the smoothing the rest; since the smoothing part copies the profile of D itself, that ratio is the *fidelity* at which the view is read. Proposition 3.6 prices it: the values Rep_{γ_R} can take span an interval of length $\frac{m}{m+\gamma_R}\left(1-\frac{m+\gamma_R}{N}\right)$, which closes to a point once $m+\gamma_R \geq N$ — at $\gamma_R = N$, for every view at once, leaving the reference attributes unread. We set instead

$$\gamma_R = \alpha N, \qquad \alpha = \tfrac{1}{100}, \tag{4}$$

reading a view at fidelity $\frac{1}{2}$ once it holds an α fraction of D and damping smaller ones, which is what smoothing is for; on Adult that is a few hundred tuples. Section 3.4 balances γ the same way, against the support of a single bin rather than of the whole view (Remark 1). Both α and N are public (Section 2.1), so the choice costs no budget and γ_R is a constant of the mechanism rather than an argument of Algorithm 1.

Proposition 3.6 (Range of smoothed representativeness). *Fix a database size N, a view size $m \leq N$ and a reference attribute A, and write $M = m + \gamma_R$. As the two A-profiles range over all non-negative vectors $(c_v^A)_v$ and $(d_v^A)_v$ with $\sum_v c_v^A = N$, $\sum_v d_v^A = m$ and $d_v^A \leq c_v^A$, the values of $Rep_{\gamma_R}^A$ fill an interval of length*

$$\frac{m}{M}\left(1-\frac{M}{N}\right) \quad \text{if } M < N, \qquad \text{and a single point otherwise.}$$

So do the values of Rep_{γ_R}, which averages such quantities over attributes whose profiles may be chosen independently of one another.

Proof. Write $x_v = c_v^A/N$ and $y_v = d_v^A/M$, so $\sum_v x_v = 1$ and $\sum_v y_v = m/M$, and put

$$S = \sum_v \min(x_v, y_v).$$

Since $\sum_v |x_v - y_v| = \sum_v x_v + \sum_v y_v - 2S$, Eq. (2) reads

$$Rep_{\gamma_R}^A = \frac{1 - m/M}{2} + S,$$

an increasing affine function of S. The admissible pairs form a convex set and S is continuous on it, so both range over an interval of the same length, and it is enough to find the two ends of the overlap S.

Above. $\min(x_v, y_v) \leq y_v$ gives $S \leq m/M$, with equality exactly when $y_v \leq x_v$ for every v. A view drawn in proportion to the database, $d_v^A = m\, c_v^A/N$, has $y_v = (m/M)\, x_v \leq x_v$ and attains it.

Below. From $d_v^A \leq c_v^A$ we get $y_v \leq c_v^A/M = (N/M)\, x_v$, so $x_v \geq (M/N)\, y_v$; and $y_v \geq (M/N)\, y_v$ whenever $M \leq N$. Hence $\min(x_v, y_v) \geq (M/N)\, y_v$ for every v, and summing, $S \geq (M/N)(m/M) = m/N$. A view sitting on a single value attains it: let one value occur exactly m times in D with all of its tuples inside the view, so that $S = \min(m/N,\ m/M) = m/N$.

So S fills $[m/N,\ m/M]$, an interval of length $\frac{m}{M} - \frac{m}{N} = \frac{m}{M}(1 - \frac{M}{N})$. If instead $M \geq N$ then $y_v \leq c_v^A/M \leq c_v^A/N = x_v$ for every v, so $S = m/M$ always and the interval is a point. □

Smoothing has a second, useful consequence: it bounds Rep_{γ_R} away from 1, which the proof of Proposition 3.11 relies on. The two vectors compared in Eq. (2) do not carry the same total mass: the database side sums to 1, while the view side sums to $m/(m+\gamma_R)$, which is below 1 because smoothing inflates the denominator without adding to the counts. Two non-negative vectors are at least as far apart in L_1 as their totals differ, so whatever the two profiles look like,

$$\sum_v \left|\tfrac{c_v}{N} - \tfrac{d_v}{m+\gamma_R}\right| \;\geq\; 1 - \frac{m}{m+\gamma_R} \;=\; \frac{\gamma_R}{m+\gamma_R},$$

and substituting this into Eq. (2) gives

$$Rep_{\gamma_R}(p, D) \;\leq\; 1 - \frac{\gamma_R}{2(m+\gamma_R)} \;\overset{\gamma_R = \alpha N}{\leq}\; \frac{2+\alpha}{2(1+\alpha)}, \tag{5}$$

for the last step, $m \leq N$ makes $m + \gamma_R \leq (1+\alpha)N$, so the fraction is at least $\alpha/(2(1+\alpha))$, and the average inherits the bound from the attributes it averages. It is attained when the view is the whole database, and it is the upper end of the interval of Proposition 3.6.

Coverage. A deviation seen on a handful of tuples is not evidence, and explanation frameworks pair a deviation measure with a mass measure: the support of association rules [3], the support threshold of MacroBase [6] and of Diff [2], the coverage of an anchor [57]. Since the object being explained is a comparison, the mass has to account for both outputs. Taking the product of the two output sizes, as a two-sided support measure suggests, has the same defect as the distributional interestingness: it vanishes on one-sided views, and it grows with the two output sizes, which is what makes a single tuple move it by an amount that grows with N (Proposition 3.10). **(AG) Small issue: The sensitivity of the product is $o_1 + o_2 + 1$, so it grows with the two output sizes, and the view size enters only as an upper bound on them. Saying it grows with the outputs would be more accurate.** We take instead the evidence the two outputs place in the view *jointly*, normalised by the public N:

$$Cov(p, Q_1, Q_2, D) = \frac{|Q_1(p(D))| + |Q_2(p(D))|}{|D|}. \tag{6}$$

Dividing by N costs nothing, since N is public (Section 2.1), and it removes the growth with the database size: *Cov* reads as a fraction of the database rather than a count, so it is comparable across datasets and across views of very different sizes. Its range is $[0, 2]$, the upper end reached only when both outputs cover all of D, and it is 0 exactly when neither output meets the view, which is the one case in which there is nothing to explain. Summing rather than taking the smaller of the two counts matters for the same reason the sum replaces the product: a view that one output fills and the other leaves empty keeps the full weight of the evidence that is there.

Diversity. When several results are returned together their marginal value depends on what has already been shown, which is the principle of maximal marginal relevance [16] and of the axiomatic

treatments that follow it [35, 44]. The aspect an explanation describes is its attribute, so we discourage reusing one, and grade the penalty by how many times it has already been used rather than charging every repeat alike:

$$Div(A, S) = \lambda^{\,\mathrm{count}(A,S)}, \qquad 0 \le \lambda \le 1, \tag{7}$$

where $\mathrm{count}(A, S)$ is the number of previously selected explanations already using attribute A. At $\lambda = 1$ the term disappears and repeats are free; at $\lambda = 0$ every repeat scores zero, which empties a candidate of value without removing it from the pool, so the mechanism may still return one. Since *Div* reads only previously selected *outputs* and never the database, it adds no data sensitivity to the selection utility.

From a sequence to a set. Equation (7) is defined relative to the explanations chosen in earlier iterations, so as written it scores an *ordered sequence*, not a set, while what we release is a set. We therefore state the objective the algorithm approximates directly.

Definition 3.7 (Set-level explanation utility). Let E be a set of candidates, and for an attribute A let $h^A_{(1)} \ge h^A_{(2)} \ge \cdots$ be the scores $HD_\gamma(c)$ of the members of E with $A_c = A$, in decreasing order, with $h^A_{(j)} = 0$ for every j beyond the size of that group. For a non-increasing weight profile $w_1 \ge w_2 \ge \cdots \ge 0$ with $w_1 = 1$,

$$F_w(E) = \sum_A \sum_{j \ge 1} w_j \, h^A_{(j)}. \tag{8}$$

(AG) A_c has not been introduced. Adding one clause to Definition 3.1, writing A_c for the attribute of a candidate c, fixes it here and in Proposition 3.8. (AG) The inner sum of Eq. (8) has no upper limit, while $h^A_{(j)}$ exists only up to the size of the attribute group. Simplest fix is to adopt the convention $h^A_{(j)} = 0$ for j beyond the group.

Within each attribute group the strongest explanation counts in full and every further one is discounted by its rank. The graded penalty of Equation (7) is the profile $w_j = \lambda^{j-1}$, and a flat rule is $w = (1, \lambda, \lambda, \dots)$. The two agree on the first two members of a group and differ only from the third onwards, so they can disagree about which set is best only when some attribute is used at least three times: that needs $k \ge 3$, and once $k > 2|Attrs|$ it is unavoidable.

Proposition 3.8 (The set objective agrees with the sequential rule). *Order E arbitrarily and write $S_{<c}$ for the members preceding c, so that the per-round rule of Eq. (7) awards E the total $\sum_{c \in E} HD_\gamma(c) \cdot Div(A_c, S_{<c})$. With $w_j = \lambda^{j-1}$, $F_w(E)$ equals that total under the ordering maximising it, and upper-bounds it under every ordering.*

Proof. Since $\mathrm{count}(A, S)$ counts only the members of S that carry A, the weight a candidate receives depends on its position within its own attribute group and not on how the groups are interleaved. The total therefore splits into independent per-group totals, and an ordering is free to choose each group's internal order. Within the group of A, an ordering that places a member j-th awards it λ^{j-1}, so the group total is $\sum_j \lambda^{j-1} h_{\sigma(j)}$ for the induced permutation σ. The weights are non-increasing, so by the rearrangement inequality this is maximised by pairing the largest weight with the largest score, that is by visiting the group in decreasing HD_γ, and the maximum is $\sum_j \lambda^{j-1} h^A_{(j)}$. Summing over groups gives Eq. (8), and no ordering gives more. The special cases $\lambda = 0$ and $\lambda = 1$ recover a group maximum and a modular sum respectively. □

Greedy selection on F_w is not a constant-factor approximation. Because F_w decomposes over attribute groups with no interaction between them, it returns the *exact* maximiser, and the per-round score of Algorithm 2 is precisely its marginal gain; Appendix 7 establishes both. **(AG) The $(1 - 1/e)$ guarantee requires F_w to be monotone submodular, which is now proved only in the appendix, so the reader meets the guarantee before the hypothesis. Please add one sentence recording that F_w is monotone submodular, with a pointer to the appendix, closes the gap.**

3.3 Sensitivity of the Measures

Private selection with the exponential mechanism (Definition 2.15) calibrates its randomness to the global sensitivity of the utility (Definition 2.13). All statements are with respect to the unbounded neighbouring relation of Definition 2.11. For a function f and a size N we write $GS_N(f)$ for the largest change of f between a database of size N and a neighbour, so that $GS(f) = \sup_N GS_N(f)$. Under Definition 2.11 a neighbour of a database of size N has size $N+1$ or $N-1$, and that asymmetry is what the two cases of Proposition 3.11 turn on. **(AG) This sentence is dense. Two things would help. The clause after the semicolon repeats what the supremum already says and can go. And it is worth stating that a neighbour of a database of size N has size $N+1$ or $N-1$, since that asymmetry is exactly what the two cases of Proposition 3.11 turn on.** A mechanism running on the private database cannot read its scale off $|D|$. Under Definition 2.11 a neighbouring pair has $|D| = N$ and $|D'| = N+1$, so such a mechanism would run at two different scales on the two databases, whereas the e^ϵ argument needs one scale common to both. It must instead fix the scale from a public $\bar{N}$ that does not depend on D and satisfies $\bar{N} \le |D|$, which is what Section 4 does; the bounded relation, under which the size is constant across the pair, would serve equally well. **(AG) Publicity of $|D|$ is necessary here but it is not sufficient. Under Definition 2.11 a neighbouring pair has $|D| = N$ and $|D'| = N+1$, so a mechanism that takes its scale from $|D|$ runs the exponential mechanism at two different temperatures on the two databases, whereas the usual e^ϵ argument needs one common scale. Two possible fixes. Either adopt the bounded relation, where N is genuinely constant across the pair, or fix the scale from a public $\bar{N}$ that does not depend on D.** Upper bounds below hold for every candidate; lower bounds and equalities are attained by explicit databases, and so require that the view and the two queries admit the tuples those databases use. We begin with the one bound that takes real work, the smoothed Histogram Distance, and then state all four results together.

Proposition 3.9 (Sensitivity of Smoothed Histogram Distance). $GS(HD_\gamma) = \dfrac{1}{1+\gamma}$.

Proof. Let $D' = D \cup \{t\}$. If t does not satisfy p nothing changes. Otherwise t falls into exactly one bin v^* of A, and every other term of Eq. (1) is unchanged, so only that term has to be bounded.

Write $n = H_{p(D),A}(v^*)$ for the support of the view in that bin and $q_j = H_{Q_j(p(D)),A}(v^*)$ for $j = 1, 2$ for the two output counts in it, and let $n' = H_{p(D'),A}(v^*)$ and $q'_j = H_{Q_j(p(D')),A}(v^*)$ be the same quantities on D'. The numerators of the term before and after are $x = |q_1 - q_2|$ and $x' = |q'_1 - q'_2|$; since $Q_j(p(D)) \subseteq p(D)$ by Proposition 2.2, $0 \le x \le n$. Each query either selects t or not, so $q'_j = q_j + \delta_j$ with $\delta_j \in \{0, 1\}$, and $n' = n + 1$ because t satisfies p and falls in v^*. Substituting these,

$$x' = |q'_1 - q'_2| = \big|(q_1 + \delta_1) - (q_2 + \delta_2)\big| = \big|(q_1 - q_2) + (\delta_1 - \delta_2)\big|,$$

whereas $x = |q_1 - q_2|$. Both x and x' are absolute values, so by the reverse triangle inequality their difference is at most the absolute value of the difference of their arguments:

$$|x' - x| \;\le\; \big|\,[(q_1 - q_2) + (\delta_1 - \delta_2)] - (q_1 - q_2)\,\big| \;=\; |\delta_1 - \delta_2| \;\le\; 1,$$

the last step because $\delta_1, \delta_2 \in \{0, 1\}$. The term therefore moves from $x/(n+\gamma)$ to $x'/(n+1+\gamma)$. If $n = \gamma = 0$ the bin is excluded from the sum on D by the restriction stated after Definition 3.4 and present in it on D', so the comparison is between an absent term and one of value $x' \le 1 = \frac{1}{1+\gamma}$. **(AG) One sentence here: At $\gamma = 0$ this bin is excluded from the sum on D by the restriction stated after Definition 3.4 and included on D', so the comparison is between a term that is absent and a term that is present. As written it reads as though the bin sat in both sums.** Otherwise put $K = (n+\gamma)(n+1+\gamma) > 0$ and bound the two directions separately. Upwards, using in turn $x' \le x + 1$, a common denominator, the cancellation of $xn + x\gamma$, then $x \ge 0$ and $n \ge 0$:

$$\begin{aligned}
\frac{x'}{n+1+\gamma} - \frac{x}{n+\gamma} &\le \frac{x+1}{n+1+\gamma} - \frac{x}{n+\gamma} \\
&= \frac{(x+1)(n+\gamma) - x(n+1+\gamma)}{K} \\
&= \frac{n+\gamma-x}{K} \\
&\le \frac{n+\gamma}{K} = \frac{1}{n+1+\gamma} \\
&\le \frac{1}{1+\gamma}.
\end{aligned}$$

Downwards, using in turn $x' \ge x - 1$, a common denominator, the same cancellation, then $x \le n$ and the step $(\star)$ below:

$$\begin{aligned}
\frac{x}{n+\gamma} - \frac{x'}{n+1+\gamma} &\le \frac{x}{n+\gamma} - \frac{x-1}{n+1+\gamma} \\
&= \frac{x(n+1+\gamma) - (x-1)(n+\gamma)}{K} \\
&= \frac{x+n+\gamma}{K} \\
&\le \frac{2n+\gamma}{K} \\
&\overset{(\star)}{\le} \frac{1}{1+\gamma}.
\end{aligned}$$

Step $(\star)$ is the only one that is not immediate. It asserts $(2n+\gamma)/K \le 1/(1+\gamma)$. Since $K > 0$ and $1 + \gamma > 0$, cross-multiplying preserves the inequality, so the claim is equivalent to $K - (2n+\gamma)(1+\gamma) \ge 0$. Expanding both products,

$$(2n+\gamma)(1+\gamma) = 2n + 2n\gamma + \gamma + \gamma^2,$$
$$K = (n+\gamma)(n+1+\gamma) = n^2 + n + 2n\gamma + \gamma + \gamma^2.$$

The terms $2n\gamma + \gamma + \gamma^2$ occur in both lines and cancel, leaving $n^2 + n$ from the second and $2n$ from the first, so

$$K - (2n+\gamma)(1+\gamma) = (n^2 + n) - 2n = n(n-1),$$

which is non-negative for every integer $n \ge 0$, vanishing only at $n = 0$ and $n = 1$. Equality holds for a bin holding one tuple of the view selected by Q_1 only, whose term is $\frac{1}{1+\gamma}$: adding a tuple of the same bin selected by Q_2 only makes the two counts equal and the term 0. □

Step $(\star)$ is an equality exactly when $n(n-1) = 0$, that is for $n \in \{0, 1\}$: a bin the new tuple populates from empty, or one already holding a single tuple of the view. For $n \ge 2$ the bound $(2n+\gamma)/\big((n+\gamma)(n+1+\gamma)\big)$ is strictly smaller, and it falls as n grows. The sensitivity is therefore dictated by the sparsest bins of any view, whereas the candidates that make good explanations rest on well-populated ones.

Proposition 3.10 (Sensitivity of the forms used in prior work). *Let N be the database size, and restrict attention to neighbouring databases on which the measure in question is defined.*

(i) Interestingness. *The distributional form has $GS \ge 1$, half of its range $[0, 2]$.*
(ii) Representativeness. *The unsmoothed form $1 - TVD$ has $GS_N \ge \frac{1}{2} - \frac{1}{N}$.*
(iii) Coverage. *The two-sided support $|Q_1(p(D))|\,|Q_2(p(D))|$ has $GS_N = 2N + 1$.*

None of the three decays with N, and the last grows with it.

Proof. (i) The score is twice a total variation distance and so lies in $[0, 2]$. Let $Q_1(p(D)) = \{t\}$ with $t[A] = v_1$, and let $Q_2(p(D))$ place all its mass on a bin $v_2 \ne v_1$, so the two conditional distributions are point masses at v_1 and at v_2 and the score is 2. Add one tuple of the view, selected by Q_1 only and with A-value v_2: the first distribution becomes uniform on $\{v_1, v_2\}$ and the score drops to 1. The rest of D is unconstrained, so a change of 1 is available at every database size.

(ii) Let D hold one tuple of the view carrying the value v_1 on every attribute of $\mathcal{A}_R(p)$, together with $N - 1$ tuples outside the view carrying v_2. The view distribution is then a point mass at v_1 while the database gives v_1 weight $\frac{1}{N}$, so the measure is $\frac{1}{N}$. Adding one tuple of the view with value v_2 makes the view distribution uniform on $\{v_1, v_2\}$ and the measure $\frac{N+3}{2(N+1)}$. The change is $\frac{N+3}{2(N+1)} - \frac{1}{N}$, which exceeds $\frac{1}{2} - \frac{1}{N}$ because $\frac{N+3}{2(N+1)} > \frac{1}{2}$.

(iii) Write $o_j = |Q_j(p(D))|$ for $j = 1, 2$; by Proposition 2.2 both are at most $m = |p(D)| \le N$. A tuple of the view selected by both queries sends $o_1 o_2$ to $(o_1 + 1)(o_2 + 1)$, a change of

$$(o_1 + 1)(o_2 + 1) - o_1 o_2 = o_1 + o_2 + 1 \;\le\; 2N + 1.$$

Equality holds at $o_1 = o_2 = m = N$, that is when the view is the whole database and both queries select all of it. □

3.4 The Two-Stage Model

A single private selection over $\mathcal{E}$ would have to score every candidate, and scoring a candidate means building the histograms of its attribute over both outputs inside its view. That is $|\mathcal{P}| \cdot |Attrs|$ pieces of work, repeated in each of the k selection rounds. Most of it is wasted: representativeness and coverage depend on the view alone, so whether a region can support an explanation at all can be decided before any attribute is looked at. The framework therefore splits the problem in two. The *view stage* scores each view by the product of the two region-level measures,

$$Use(p, Q_1, Q_2, D) = Rep_{\gamma_R}(p, D) \cdot Cov(p, Q_1, Q_2, D), \tag{9}$$

with Rep_{γ_R} from Definition 3.5 and Cov from Eq. (6), and retains the τ best. The *explanation stage* then scores only the candidates built from the retained views, by the product of the two candidate-level measures:

$$\widetilde{U}_i(D; c) = HD_\gamma(D; p, A, Q_1, Q_2) \cdot Div(A, S_{i-1}) \tag{10}$$

for a candidate $c = (p, A)$. **(AG) The workflow paragraph and Figure 1 say γ is refreshed each round from the accountant, so this utility and Proposition 3.12 are really stated for the round's γ_i. Suggest carrying the index here, or adding a clause that the bound holds for whichever γ the round uses.** This reduces the candidate-level work from $|\mathcal{P}| \cdot |Attrs|$ to $\tau \cdot |Attrs|$, a factor $|\mathcal{P}|/\tau$, in every round.

Efficiency is the visible reason for the split, but not the binding one. A single-stage mechanism would have to select on the product of all four measures, and a product is worse than its factors: by the product rule its sensitivity picks up a term in which the movement of one factor is multiplied by the *level* of the other, and the levels here are not small. Coverage alone can reach 2, so an interestingness score that moves by Δ drags the combined score by 2Δ before the coverage term is even counted. Splitting removes that cross term entirely. Each stage is its own private mechanism, calibrated to the sensitivity of its own utility, and neither pays for the other's range. Propositions 3.11 and 3.12 are what the two stages pay, and Section 4.2 composes them into ϵ.

Proposition 3.11 (Sensitivity of View Usefulness). *Let $\gamma_R = \alpha N$ for a public $\alpha \in [0, 1]$, as in Eq. (4). Then*

$$GS_N(Use) \;\le\; \frac{4}{N}.$$

Proof. Write $m = |p(D)|$ for the view size, $M = m + \gamma_R$ for the smoothed view size, and $s = |Q_1(p(D))| + |Q_2(p(D))|$ for the evidence the two outputs place in the view, so that $0 \le s \le 2m \le 2N$ by Proposition 2.2 and $Cov = s/N$. For a reference attribute A let $x_v^A = c_v^A/N$ and $y_v^A = d_v^A/M$ be the two profiles it compares, and write $\theta = \sum_v y_v^A = m/M$, the same for every A. A prime marks a quantity on a neighbour of size $N + 1$, and Δ a primed quantity minus its unprimed one. We bound the change to such a neighbour first and collect the two directions at the end. So let $D' = D \cup \{t\}$, let $v(t)$ be the A-value of t, and set

$$w^A = \Delta y^A - \Delta x^A,$$

the relative movement of the two profiles.

Step 1: the two factors. Since $Use = Rep_{\gamma_R} \cdot Cov$ and $\Delta(Rep_{\gamma_R} Cov) = Cov\,\Delta Rep_{\gamma_R} + Rep'_{\gamma_R}\Delta Cov$,

$$|\Delta Use| \;\le\; Cov\,|\Delta Rep_{\gamma_R}| + Rep'_{\gamma_R}\,|\Delta Cov|. \tag{11}$$

Step 2: representativeness moves no more than the profiles do. By Eq. (2), $Rep^A_{\gamma_R} = 1 - \frac{1}{2}\|x^A - y^A\|_1$, so the reverse triangle inequality for the L_1 norm gives

$$\begin{aligned} |\Delta Rep^A_{\gamma_R}| &= \tfrac{1}{2}\Big|\,\|x'^A - y'^A\|_1 - \|x^A - y^A\|_1\Big| \\ &\le \tfrac{1}{2}\big\|(x'^A - y'^A) - (x^A - y^A)\big\|_1 \;=\; \tfrac{1}{2}\|w^A\|_1. \end{aligned}$$

What matters is that w^A stays whole: when the view mirrors the database the arrival of t rescales both profiles in step and w^A nearly cancels, which bounding $\|\Delta y^A\|_1$ and $\|\Delta x^A\|_1$ apart would hide.

Step 3: how the profiles move. Both cases are one computation. Write $\Lambda = M' - M \ge 0$ for the growth of the smoothed view size and

$$\kappa = \frac{\Lambda\,(N+1)}{M'}.$$

Away from $v(t)$ neither count changes and only the denominators move, which gives

$$w_v^A = \frac{c_v^A}{N(N+1)} - \frac{d_v^A \Lambda}{MM'} = \frac{x_v^A - \kappa\, y_v^A}{N+1}.$$

At $v = v(t)$ the arrival of t adds a further ρ to this, where

$$\rho = \begin{cases} -1/(N+1), & t \text{ outside the view,} \\ 1/M' - 1/(N+1), & t \text{ inside.} \end{cases}$$

Both κ and ρ are the same for every A. Since x^A and y^A are non-negative with $\sum_v x_v^A = 1$ and $\sum_v y_v^A = \theta$, we have $\sum_v |x_v^A - \kappa y_v^A| \le \sum_v x_v^A + \kappa \sum_v y_v^A = 1 + \kappa\theta$, so summing the two displays over v and averaging over $\mathcal{A}_R(p)$,

$$|\Delta Rep_{\gamma_R}| \;\le\; \tfrac{1}{2}\|w\|_1 \;\le\; \frac{1+\kappa\theta}{2(N+1)} + \frac{|\rho|}{2}, \tag{12}$$

where $\|w\|_1$ denotes the average of $\|w^A\|_1$ over $\mathcal{A}_R(p)$. Recall also $Cov \le 2m/N$ and $Rep'_{\gamma_R} \le \frac{2+\alpha}{2(1+\alpha)}$ from Eq. (5).

Step 4: t falls outside the view. Then m, s and every d_v^A stay put, $\Lambda = \alpha$ and $M' = M + \alpha$, while $|\rho| = 1/(N+1)$ and $\Delta Cov = -s/(N(N+1))$ gives $|\Delta Cov| \le 2m/(N(N+1))$. Putting these into Eqs. (12) and (11) and collecting over the common factor $m/(N(N+1))$, which uses $2 + \frac{2+\alpha}{1+\alpha} = \frac{4+3\alpha}{1+\alpha}$,

$$|\Delta Use| \;\le\; \frac{m}{N(N+1)} \cdot \frac{4+3\alpha}{1+\alpha} \;+\; \frac{\alpha\, m^2}{N\,M\,M'},$$

the second term being $Cov \cdot \kappa\theta/\big(2(N+1)\big)$ written out. Both increase in m, so both are largest at $m = N$, where the view is all of D:

$$|\Delta Use| \;\le\; \frac{4+3\alpha}{(1+\alpha)(N+1)} + \frac{\alpha}{(1+\alpha)\big(N(1+\alpha)+\alpha\big)}.$$

Subtracting this from $4/N$ leaves

$$\frac{4}{N(N+1)} + \frac{\alpha\big[\alpha(N+1) - 1\big]}{(1+\alpha)(N+1)\big(N(1+\alpha)+\alpha\big)},$$

and the second term is at least $-1/\big(N(N+1)\big)$, since $\alpha \le 1$ and $N(1+\alpha)+\alpha \ge N$. The difference is therefore at least $3/\big(N(N+1)\big) > 0$.

Step 5: t falls inside the view. Now m becomes $m+1$ and $d^A_{v(t)}$ becomes $d^A_{v(t)}+1$, while s becomes $s+\delta$ with $\delta \in \{0,1,2\}$. The range of δ is the number of the two queries that select t: an output is $\sigma_\varphi(D)$ (Section 2.1), decided tuple by tuple, so t cannot change whether any other tuple is selected. Here $\Lambda = 1+\alpha$ and $|\rho| = |1/M' - 1/(N+1)|$, while $|\Delta Cov| = |\delta N - s|/(N(N+1))$ is at most $\max(2N-s,\, s)/(N(N+1))$, which ties it to $Cov = s/N$ rather than letting both reach their extremes at once. Writing $\beta = m/N$, Eqs. (12) and (11) are again explicit in β and Cov, and the maximum of $(N+1)|\Delta Use|$ over $\beta \in [0,1]$ and $Cov \in [0, 2\beta]$ is 3.99 at $\alpha = \frac{1}{100}$ and 3.95 at $\alpha = \frac{1}{20}$. It rises as α falls, approaching 4 without reaching it.

Step 6: the two directions. A neighbour of a database of size N has size $N+1$ or $N-1$. Steps 4 and 5 bound the change to a size-$(N+1)$ neighbour by $4/(N+1)$. Run at base size $N-1$ they bound the change to a size-$(N-1)$ neighbour by $4/N$: the computation is unchanged, because $\gamma_R = \alpha n$ leaves α fixed as the base size n varies, and every bound above carries $n+1$ in its denominator. The second is the larger of the two, and it is what fixes the constant. □

Proposition 3.12 (Sensitivity of the selection utility). $GS(\widetilde{U}_i) \le 1/(1+\gamma)$.

Proof. Condition on the history S_{i-1} of previously released outputs, which is legitimate under adaptive composition. Given that history, $Div(A, S_{i-1})$ is a constant in $[0,1]$ determined entirely by released values, so it acts as a bounded multiplier on the data-dependent factor and cannot increase its sensitivity. The claim follows from Proposition 3.9. □

Filtering is not free. The view stage ranks by *Use*, which no explanation score enters, so a view of middling usefulness may host the single most interesting explanation and still be dropped; and because the filter itself is private, the retained set is not even the exact top-τ by *Use*. We do not bound what this costs. The one-shot top-τ mechanism admits a high-probability utility guarantee whose slack grows as $\tau\Delta \log|\mathcal{P}|/\epsilon_P$, which at $\Delta = 4/N$ is $4\tau \log|\mathcal{P}|/(N\epsilon_P)$ and so vanishes with the database size. What that slack must be compared against, however, is not the scale of *Use* but the gap between the τ-th and the $(\tau{+}1)$-th best view, and that gap narrows as the view space grows denser. The guarantee is therefore informative only under a lower bound on the gap, which we do not assume. What the form does say is that the cost of filtering grows linearly in τ, which is an argument for keeping τ small.

4 ALGORITHM

Section 3 fixed the measures the algorithm optimises and the two stages it runs them in. This section turns that into code. Algorithm 1 is the entry point: it splits the budget, privately selects a small set of views, and hands them to Algorithm 2, which selects the explanations and releases their histograms. The same separation between a quality definition and the private mechanism that optimises it is used by other differentially private explanation frameworks [63]. What is left to specify is how the two remaining constants are fixed, what each line of the algorithms does, and what the pipeline costs in privacy and in time.

4.1 Explanation Generation

Choosing τ and γ. Two constants are left once the budget is split: how many views the first stage keeps, and how hard the second stage smooths. Neither is something an analyst can reasonably be asked to guess, so the algorithm fixes both itself.

The k explanations carry k distinct attributes but sit on at most k views, so $\tau = k$ is the smallest value at which the optimal set is representable. It is not sufficient, and the reason is not noise: the view stage orders by *Use*, into which no explanation score enters, so the views hosting the best pairs need not be the k best by *Use* (Section 3.4). Against that, the one-shot top-τ mechanism perturbs each score at a scale proportional to $\tau\, GS(Use)/\epsilon_P$, so every extra view is paid for *linearly* while the benefit saturates. The floor scales with k and the price grows linearly above it, which fixes the form $\tau = ck$; the constant is a parameter of the framework rather than one we derive, and we take $c = 3$:

$$\tau = \min\big(3k,\ |\mathcal{P}|\big). \tag{13}$$

For γ, the exponential mechanism supplies the scale. Selecting among $|C_i| = \tau|Attrs|$ candidates with budget ϵ_i and sensitivity $GS(HD_\gamma) = 1/(1{+}\gamma)$, it separates two candidates whose utilities differ by about $2GS(HD_\gamma)\ln|C_i|/\epsilon_i$. A well-supported bin contributes close to 1 to HD_γ, so one unit of utility is the natural scale to resolve; setting that quantity to 1 gives

$$\gamma = \frac{2\ln(\tau\,|Attrs|)}{\epsilon_i}, \qquad \epsilon_i = \epsilon_S/k. \tag{14}$$

Both rules read only k, $|\mathcal{P}|$, $|Attrs|$ and the budget, all of them public, so by post-processing (Theorem 2.19) they cost nothing.

Remark 1 (The range of γ). Eq. (14) is a floor, and the measure imposes a ceiling against it. In bin v a candidate contributes $\delta_v/(n_v{+}\gamma)$, where $n_v = H_{p(D),A}(v)$ is the support of the view in that bin and $\delta_v \le n_v$ the disagreement between the two outputs. While $\gamma \ll n_v$ this is the *relative* disagreement δ_v/n_v, which is what the measure is meant to report; once $\gamma \gg n_v$ it degenerates to the absolute count δ_v/γ and the bin is silenced however completely it changed. Writing $m = |p(D)|$ and b for the number of bins of A, the largest attainable score is $\sum_v n_v/(n_v+\gamma)$, which for support spread evenly is $bm/(m+b\gamma)$: within a factor of two of b while $\gamma \le m/b$, and decaying as m/γ past it. The usable range is therefore

$$\frac{2\ln(\tau|Attrs|)}{\epsilon_i} \ \le\ \gamma \ \le\ \frac{m}{b}, \tag{15}$$

non-empty exactly when $|p(D)| \ge b \cdot 2\ln(\tau|Attrs|)/\epsilon_i$. That is the resolution of the framework: a view smaller than its number of bins times the separation the mechanism can achieve carries no visible explanation at any γ, since no value both resolves the candidates and keeps the score relative.

Algorithm 1. Line 1 splits the budget three ways, between view selection, explanation selection and histogram release. The split is a parameter, and we give the three stages equal shares. Line 3 fixes the three constants: γ_R by Eq. (4), and τ and γ by Eqs. (13) and (14). All three are functions of public quantities. Line 4 scores every candidate view by *Use*, which reads the database but no attribute. Line 5 is the only private step of this algorithm: it draws the τ best views with the one-shot top-τ mechanism of Definition 2.16,

Algorithm 1 Private-Queries-Diff-Top-k-Explanation

Require: Private dataset D; queries Q_1, Q_2; views $\mathcal{P}$; explanation attributes $Attrs \subseteq Attr(D)$; explanations k; diversity λ; privacy budget ϵ

Ensure: k differentially private explanation histograms

1: $\epsilon_P, \epsilon_S, \epsilon_H \leftarrow \epsilon/3,\ \epsilon/3,\ \epsilon/3$ ▷ see text
2: /* **Private view selection** */
3: $\gamma_R \leftarrow \alpha N;\ \tau \leftarrow \min(3k, |\mathcal{P}|);\ \gamma \leftarrow 2\ln(\tau|Attrs|)\, k/\epsilon_S$
4: $u_{use}(p) \leftarrow Use(p, Q_1, Q_2, D)$
5: $\widehat{\mathcal{P}} \leftarrow$ OneShotTop-$\tau(\mathcal{P}, u_{use}, \epsilon_P, \Delta{=}\frac{4}{|D|})$ ▷ ϵ_P-DP
6: $\mathcal{E}_{\text{out}} \leftarrow$ Find-Top-k-Expl$(D, Q_1, Q_2, \widehat{\mathcal{P}}, Attrs, k, \gamma, \lambda, \epsilon_S, \epsilon_H)$
7: **return** $\mathcal{E}_{\text{out}}$

Algorithm 2 Find-Top-k-Explanations

Require: D; Q_1, Q_2; selected views $\widehat{\mathcal{P}}$; attributes $Attrs$; k; smoothing γ; diversity λ; budgets ϵ_S, ϵ_H

$Counts \leftarrow \emptyset;\ \mathcal{E}_{\text{out}} \leftarrow \emptyset;\ \epsilon_i \leftarrow \epsilon_S/k$
for $i \leftarrow 1$ to k **do**
  $C_i \leftarrow \{(p, A) \in \mathcal{E} : p \in \widehat{\mathcal{P}},\ (p, A, \cdot, \cdot) \notin \mathcal{E}_{\text{out}}\}$
  $u_i(p, A) \leftarrow HD_\gamma(D; p, A, Q_1, Q_2) \cdot \lambda^{Counts[A]}$
  $(p_i, A_i) \leftarrow$ ExpMech$(C_i, u_i, \epsilon_i, \Delta{=}\frac{1}{1+\gamma})$
  $H_i^1 \leftarrow H_{Q_1(p_i(D)),A_i};\ H_i^2 \leftarrow H_{Q_2(p_i(D)),A_i}$
  $\hat{H}_i^j \leftarrow H_i^j + \text{Laplace}(2k/\epsilon_H)$ **for** $j = 1, 2$
  $\mathcal{E}_{\text{out}} \leftarrow \mathcal{E}_{\text{out}} \cup \{(p_i, A_i, \hat{H}_i^1, \hat{H}_i^2)\};\ Counts[A_i]{+}{+}$
end for
return $\mathcal{E}_{\text{out}}$

calibrated to the sensitivity $4/|D|$ of Proposition 3.11. Pairing the surviving views with the explanation attributes touches no data beyond that output, so it is post-processing and costs nothing. Line 6 passes the surviving views, the two constants and the two remaining budgets to Algorithm 2, whose output is returned unchanged.

Algorithm 2. Line 1 initialises the per-attribute counters and gives each of the k rounds an equal share ϵ_S/k of the selection budget. Line 3 forms the candidates of this round from every retained view and every explanation attribute, excluding anything already released and, by Definition 3.1, any attribute the view itself constrains. The whole retained set is used in every round: narrowing it would risk excluding the best remaining candidate, while the price the exponential mechanism pays for a larger pool is only logarithmic in its size. Line 4 scores each candidate by its smoothed histogram distance, discounted by λ raised to the number of times its attribute has already been used. Line 5 selects one candidate with the exponential mechanism at sensitivity $1/(1+\gamma)$, by Proposition 3.12. Lines 6 and 7 build the two histograms of the selected candidate and perturb each of them with Laplace noise of scale $2k/\epsilon_H$. Line 8 records the released pair and increments the counter of the attribute just used, which is what makes the next round discount it.

The algorithms on the running example. Take the refinement of Example 2.3 on Adult: $N = 48{,}842$, $|\mathcal{P}| = 1{,}052$, $|Attrs| = 10$, $k = 5$,

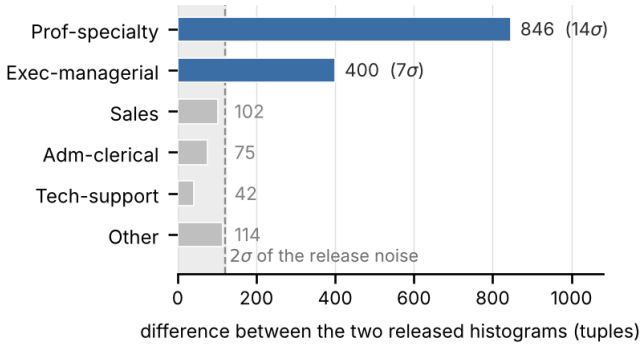


Figure 3: What the release leaves of the first explanation of the running example: the per-bin difference between the two noisy histograms of Figure 2(a). At $2k/\epsilon_H = 30$ that difference has standard deviation 60 per bin. Two bins clear twice it; the other four do not.

$\lambda = \frac{1}{2}$ and $\epsilon = 1$, so each stage receives 1/3 and Eqs. (13) and (14) give $\tau = 15$ and $\gamma \approx 150$. Algorithm 1 scores the 1,052 views by *Use* and keeps 15. The view `education = Masters` survives on a coverage of 0.068 and a representativeness of 0.80, against the 0.92 that Eq. (5) allows a view of its size. At $\gamma_R = N$ it scored 0.53, which is that ceiling exactly: the measure read none of the reference attributes and would have returned 0.53 whatever they held. The 0.12 it now falls short by is the distributional signal Eq. (4) leaves in.

Algorithm 2 pairs it with each attribute, and the first round selects `occupation`, whose two histograms are those of Figure 2(a). Figure 3 shows what survives their release: the explanation is legible exactly where it is large, and the four small bins are returned as noise. The counter of `occupation` is then incremented, and with $\lambda = \frac{1}{2}$ every remaining candidate on that attribute is scored at half its value in the next round, which is what moves the second explanation onto a different attribute.

Exactness of the second stage. Because γ is fixed once and the diversity factor depends only on attributes already released, the score of line 4 is the marginal gain of its candidate in the set objective of Definition 3.7, so selecting the highest-scoring candidate is one step of greedy on it. On that objective greedy is not an approximation but exact, because it separates over attribute groups and each group contributes a concave function of how many of its candidates are taken; Proposition 7.1 in Appendix 7 states both halves. The optimality is over the candidates the view stage admits rather than over $\mathcal{E}$; what the filter discards is the cost Section 3.4 leaves open. This is what makes a non-private greedy run a usable reference point: with the candidate set and γ held fixed it is the optimum, so the gap the private algorithm shows against it is attributable to the mechanism alone.

4.2 Privacy and Cost

Proposition 4.1 (Sensitivity of the released histograms). *Fix a candidate (p, A) and write $H^j = H_{Q_j(p(D)),A}$ for $j = 1, 2$. The*

pair (H^1, H^2) *has* L_1*-sensitivity at most* 2*, and the difference* $H^1 - H^2$ *has* L_1*-sensitivity at most* 1.

Proof. Since Q_1 and Q_2 are selections, one added tuple joins at most one bin of A, and it may be selected by both queries, by one of them, or by neither. It therefore changes each of the two histograms in at most one coordinate and by at most 1, which gives 2 for the pair. For the difference, the affected coordinate changes by $d_1 - d_2$ with $d_1, d_2 \in \{0, 1\}$, so by at most 1, and all other coordinates are unchanged. □

Releasing the pair therefore calls for Laplace$(2k/\epsilon_H)$, as in Algorithm 2; the variant that releases only the difference uses Laplace(k/ϵ_H) for the same budget.

Proposition 4.2 (Privacy of the framework). *Algorithm 2 satisfies* $(\epsilon_S + \epsilon_H)$*-differential privacy, and, with* $N = |D|$ *public as in Proposition 3.11, Algorithm 1 satisfies* ϵ*-differential privacy.*

Proof. Take Algorithm 2 first. Each round invokes the exponential mechanism with budget ϵ_S/k on a utility of sensitivity $1/(1+\gamma)$ (Proposition 3.12), which is (ϵ_S/k)-DP, and the Laplace mechanism with budget ϵ_H/k on a pair of histograms of L_1-sensitivity 2 (Proposition 4.1), which is (ϵ_H/k)-DP. Both calibrations are legitimate because γ is a constant of the mechanism rather than a function of the input: by Eq. (14) it is determined by k, $|\mathcal{P}|$, $|Attrs|$ and the budget, all of which are fixed before the data is read, so the sensitivity bound it induces holds uniformly over all databases. The rounds are adaptive, since u_i depends on the attributes already selected, but adaptive sequential composition applies [43] and gives $k(\epsilon_S/k + \epsilon_H/k) = \epsilon_S + \epsilon_H$. The conditioning covers the candidate set as well as the utility: C_i is fixed by $\widehat{\mathcal{P}}$ and by what has already been released, both of them outputs of earlier steps, so for a fixed history it is the same set on neighbouring databases.

For Algorithm 1, view selection is ϵ_P-DP by Theorem 2.17 together with the sensitivity $4/|D|$ of Proposition 3.11, which the algorithm passes to the mechanism. This is the one place the hypothesis on N is used, and it is used for the calibration rather than for the bound: Proposition 3.11 already covers neighbours of different sizes, but the theorem being invoked compares a *single* mechanism on two inputs, and a Δ recomputed from each input would make the runs on D and on D' two different mechanisms. Fixing $\Delta = 4/\bar{N}$ from a public $\bar{N} \le |D|$ that does not depend on D removes that. Candidate construction is post-processing. Sequential composition over the two stages gives $\epsilon_P + \epsilon_S + \epsilon_H = \epsilon$, so the pipeline as a whole is ϵ-differentially private. □

Complexity. Let $|\mathcal{P}|$ be the number of candidate views, $a = |Attrs|$ the number of explanation attributes, $r = |\mathcal{A}_R|$ the number of reference attributes, b the maximum number of bins per attribute, and $N = |D|$. Computing *Use* for every view costs $O(|\mathcal{P}|\, r\, N)$ by the direct route of one pass over D per view, since Rep_{γ_R} averages over the reference attributes and every view needs its own histogram on each of them; the one-shot top-τ mechanism adds $O(|\mathcal{P}| \log \tau)$, one Gumbel draw per view and a heap of size τ. Materialising the bin vectors of the τ retained views over the a explanation attributes costs $O(\tau a N)$. Scoring is then paid once rather than once per round. Because γ is fixed by Eq. (14), $HD_\gamma(p, A)$ is the same number in every round, so the τa values are computed once in $O(\tau a b)$; a round only rescales each of them by its diversity factor, samples, and perturbs the two histograms of the winner, which is $O(\tau a + b)$. The total is

$$O(|\mathcal{P}|\, r\, N + \tau a\, N + \tau a b + k(\tau a + b)) = O\big(\big(|\mathcal{P}|\, r + \tau a\big) N\big),$$

because $b \le N$ and $k \le N$. The two data passes therefore absorb everything else, and the k selection rounds, the only part that spends privacy budget, cost $O(k(\tau a + b))$ and are independent of N. Of the two passes only the first is independent of k as well as of ϵ; the second carries $\tau = \min(3k, |\mathcal{P}|)$.

Table 2: Datasets, the setting each one is used for, and where its query pairs come from.

Dataset	N	$\lvert Attrs\rvert$	$\lvert\mathcal{P}\rvert$	Setting	Pairs	Pairs from
Adult [9]	48,842	10	1,048	refinement	7	Sec. 1, ours
ACSIncome [23]	1,664,500	10	914	refinement	10	Erica [41]
Tax [12]	200,000	9	1,792	refinement	4	draft, ours
Tax-dirty [45]	200,000	9	1,977	CQA	3	rules of [54]
Hospital [56]	1,000	10	502	CQA	3	draft, rule
IPUMS-CPS [63]	1,146,552	8	1,070	contrast	17	DPXPlain [63]

5 EXPERIMENTS

We evaluate the algorithm of Section 4 on six real datasets and 44 query pairs that cover the three settings of the paper: query refinement, consistent query answering, and contrasts between two subpopulations. Wherever possible the data, the constraints and the queries are those of earlier work on these problems, so that the pairs we explain are pairs that a refinement system, a repair benchmark or a private explanation system has already been run on.

(Alon) All numbers in this section were produced with the constants of the previous draft, $\gamma_R = N$ and the view-stage sensitivity $\Delta = 7/(4|D|)$. They must be re-run under Eq. (4), $\gamma_R = N/100$, and $\Delta = 4/|D|$ of Proposition 3.11. The code for the new constants is ready (commit 70cfa6f of the experiment repository); running times do not depend on the two constants, accuracy does.

5.1 Experimental Goals

RQ1 – Use cases. Do the released explanations tell an analyst something about the difference between Q_1 and Q_2 that the two queries and the constraint do not already say?

RQ2 – Quality. How close is the private algorithm to non-private references, how does that depend on ϵ and γ, and which of the two stages limits it?

RQ3 – Scalability. How does the end-to-end running time depend on k, N, ϵ, $|\mathcal{P}|$, τ and γ, and what does privacy cost in time compared with the non-private baselines?

5.2 Experimental Setup

5.2.1 Datasets. Table 2 lists the datasets. Numeric attributes are bucketed into six quantile bins and views are conjunctions of at most two atomic conditions ($\ell = 2$); $|\mathcal{P}|$ counts every such view with at least one tuple.

Query refinement. *ACSIncome* is the Folktables replacement of Adult [23] and the dataset of the experiments of Erica [41, 42]. We use the same extract, the four queries Q_1–Q_4 and the constraint sets C_1–C_9 of that paper, and take as Q' one of the minimal refinements Erica itself returned, as stored with its artifacts; we decode the stored refinement vectors and check on the data that every decoded Q' satisfies its constraints and that Q violates them. Ten of the twelve (query, constraint set) combinations pass this check.[1] When Erica returned several minimal refinements we keep the one whose effect is least visible from the query and the constraints: the largest change in the distribution of attributes that neither mentions. *Adult* contributes the running example of Section 1 (A2) and the occupation-broadening pair of the earlier draft (A1), and five pairs of our own whose Q' we obtained with an Erica-style search for a minimal refinement under a gender, race or age representation constraint. *Tax* is the synthetic tax generator of the data-cleaning literature [12]; we use the clean 200,000-tuple file distributed with Raha [45], the marital-status refinement of the earlier draft (T1), and three refinement pairs of our own under constraints on `HasChild` and on the share of non-zero tax rates.

Consistent query answering. For CQA we need a database that actually violates its constraints. *Tax-dirty* is the dirty version of the same Tax file from Raha [45], with the Tax rules distributed with the repair benchmark of Ni et al. [54]: `Zip` → `City`, `Zip` → `State`, `FName` → `Gender`, (`Phone`, `Zip`) → `AreaCode` and three rules on exemptions. We drop one rule of that benchmark, `AreaCode` → `State`: it was written for cell-level repair, and under the subset-repair semantics of Definition 2.7 it puts 82% of the tuples in conflict (every area code that occurs with a single misspelled state), which leaves almost nothing to answer. With the remaining seven rules 3.9% of the tuples are in conflict, very unevenly: 52% in Alaska, 29% in Alabama, at most 13% elsewhere. *Hospital* is the dirty hospital file of HoloClean [56] with the dependencies `HospitalName` → `ZipCode` and `HospitalName` → `PhoneNumber`, which are among the constraints HoloClean distributes, and `ZipCode` → `City`. In both datasets $Q_\Sigma(D)$ is computed with the conflict hypergraph [19]: for a selection query it is $Q(D)$ without the tuples that take part in a conflict. We pick each CQA pair by one rule: a selection whose consistent answer drops between 10% and 90% of $Q(D)$, and drops it unevenly across the other attributes, since a uniform loss leaves nothing to explain. On Tax-dirty the selections are US Census divisions (C1–C3); on Hospital, H1 is the pair of the earlier draft (`HospitalType` = acute care) and H2 and H3 are the selections on `EmergencyService` and `HospitalOwner` that the rule picks.

Contrasts. Explaining how two subpopulations differ is the classical problem of contrast-set and emerging-pattern mining [8, 24], and it is the question DPXPlain answers privately for an aggregate [63]. We use the IPUMS-CPS extract as prepared by DPXPlain (1,146,552 tuples) and all 17 of its questions: its default question (`SEX` = `Male` against `Female`) and its control questions on `RELATE`, `MARST`, `RACE`, `CITIZEN`, `CLASSWKR`, `AGE` and `EDUC`. The attribute that defines the two groups is removed from the views and from the explanation attributes of its pair: a view on it contains only one of the two outputs, and a histogram over it says only that the groups are different groups.

Where each pair comes from. Of the 44 pairs, 27 come from earlier work (ten from Erica, seventeen from DPXPlain), four from the earlier draft of this paper (A1, A2, T1, H1), eight from our own refinement search (A3, R1–R4, T2, T3, R5) and five from our CQA selection rule (C1–C3, H2, H3). Table 3 lists all of them with their constraints and output sizes.

The four pairs of the case studies (bold in Table 3) cover the three settings and the three sources of pairs: the running example A2, Erica's Q_1C_1 on ACSIncome, the CQA pair C1 and DPXPlain's default question I1.

5.2.2 Parameters. Every experiment fixes the parameters it does not study to one default, chosen to be the realistic deployment rather than the favourable one: the whole dataset, every public view, $k = 5$, $\lambda = 0.5$, $\epsilon = 1$ split equally between the three stages, and τ and γ from Eqs. (13) and (14), which give $\tau = 15$ and $\gamma \approx 150$ on every dataset. When an experiment varies k, ϵ or τ, the two derived constants follow their rules, because that is what the algorithm does; only the γ experiment overrides Eq. (14), by a multiplier. Accuracy figures average 40 private runs per point: the run-to-run standard deviation of the set score is often comparable to its mean, and fewer runs do not separate neighbouring configurations. Every timed run is end to end, from the raw data to the released histograms, and is stopped after 60 s; a stopped run is reported as a timeout. The running-time sweeps use two to four pairs per dataset (17 in total) and the per-method comparison 19, since running time hardly depends on the pair. All experiments ran single-threaded in Python with NumPy on a 4-core cloud machine with 16 GB of memory.

5.3 Compared Methods

No earlier system explains the difference between two query outputs privately, so, as in DPXPlain, we compare against references that each remove one ingredient of the algorithm.

B1 — **no noise.** The same algorithm with every mechanism made deterministic ($\epsilon \to \infty$ in the mechanisms, while τ and γ keep their values). The gap to B1 is the cost of the noise alone.

B2 — **exhaustive optimum.** Takes the exact top-τ views by *Use*, scores every size-k subset of their candidates with F_λ, and returns the best. The full space is too large ($\binom{150}{5} \approx 6 \times 10^8$ subsets on Adult), but the optimum lies within the top k candidates of each attribute: a set that used a candidate outside the top k of its attribute could swap in an unused higher-scoring candidate of the same attribute without lowering F_λ. The pool therefore has $k\,|Attrs|$ candidates, and B2 enumerates $\binom{k|Attrs|}{k}$ subsets.

B3 — **non-private greedy.** The exact top-τ views by *Use*, then k greedy rounds with no noise.

OptPrivExpl. The private algorithm with a different computation of the statistics, used only in the running-time experiments: the counts of all views that constrain the same set of attributes are computed in one grouped pass, and the explanation histograms of the retained views in one combined pass. It releases exactly what the private algorithm releases with

[1] For Q_1C_3 and Q_4C_9 the order of the flags in the stored vectors is ambiguous, and no reading of it gives refinements that satisfy the constraints, so we leave them out.

Table 3: The 44 query pairs of the experiments, by dataset. Bold: the case studies of Section 5.4. $|Q_1| \to |Q_2|$: output sizes ($|Q_1|$ / $|Q_2|$ for contrasts). ACS codes are shown by name; SCHL 16 = high-school diploma, 19 = some college, 21 = bachelor's, 22 = master's, 23 = professional degree.

ID	Q_1	Q_2	Constraint	$\lvert Q_1\rvert \to \lvert Q_2\rvert$
Adult [9]. A1 and A2 (the running example) from the earlier draft; A3 and R1–R4: Q' from our refinement search.				
A1	occupation = Craft-repair ∧ hoursPerWeek ≥ 40	hoursPerWeek ≥ 40	≥25% gender = Female	5,447→37,155
A2	education IN (Bachelors, Masters) ∧ hoursPerWeek ≥ 50	education = Masters ∧ hoursPerWeek ≥ 30	≥25% gender = Female	3,055→2,453
A3	maritalStatus = Married-civ-spouse ∧ hoursPerWeek ≥ 45	maritalStatus IN (Divorced, Married-civ-spouse, Never-married) ∧ hoursPerWeek ≥ 38	≥25% gender = Female	8,216→35,446
R1	education IN (Masters, Doctorate, Prof-school) ∧ hoursPerWeek ≥ 50	education IN (Doctorate, Masters, Prof-school, Some-college) ∧ hoursPerWeek ≥ 35	≥30% gender = Female	1,554→12,064
R2	occupation = Prof-specialty ∧ maritalStatus = Married-civ-spouse	occupation IN (Other-service, Prof-specialty) ∧ maritalStatus IN (Married-civ-spouse, Never-married)	≥15% race IN (Black, Asian-Pac-Islander, Amer-Indian-Eskimo, Other)	3,182→8,561
R3	maritalStatus = Married-civ-spouse ∧ age ≥ 40	maritalStatus IN (Divorced, Married-civ-spouse, Widowed) ∧ age ≥ 61	≥30% gender = Female	12,932→3,296
R4	occupation = Prof-specialty ∧ hoursPerWeek ≥ 50	occupation IN (Other-service, Prof-specialty) ∧ hoursPerWeek ≥ 30	≥25% age < 30	1,681→9,186
ACSIncome [23]. Queries Q_i, constraint sets C_j and refined queries Q' of Erica [41]; Q' is one of the minimal refinements Erica returned.				
Q_1C_1	WKHP ≥ 40 ∧ SCHL ≥ 19 ∧ COW IN (local-gov, state-gov, federal-gov)	WKHP ≥ 40 ∧ SCHL ≥ 19 ∧ COW IN (local-gov, state-gov, federal-gov, private)	≥48,884 SEX = F ∧ MAR = married; ≥15,719 RAC1P = Black	141,100→587,712
Q_1C_2	WKHP ≥ 40 ∧ SCHL ≥ 19 ∧ COW IN (local-gov, state-gov, federal-gov)	WKHP ≥ 41 ∧ SCHL ≥ 19 ∧ COW IN (local-gov, state-gov, federal-gov)	≤15,249 SEX = M ∧ RELP = spouse; ≤11,498 age_bucket = 30-60 ∧ MAR = divorced	141,100→48,096
Q_2C_1	WKHP ≤ 40 ∧ SCHL ≤ 19 ∧ PINCP_bucket IN (<20K, 20K-40K)	WKHP ≤ 40 ∧ SCHL ≤ 19 ∧ PINCP_bucket IN (<20K, 20K-40K, 40K-60K)	≥125,492 SEX = F ∧ MAR = married; ≥69,207 RAC1P = Black	539,089→635,774
Q_2C_2	WKHP ≤ 40 ∧ SCHL ≤ 19 ∧ PINCP_bucket IN (<20K, 20K-40K)	WKHP ≤ 39 ∧ SCHL ≤ 19 ∧ PINCP_bucket IN (<20K, 20K-40K)	≤33,674 SEX = M ∧ RELP = spouse; ≤36,921 age_bucket = 30-60 ∧ MAR = divorced	539,089→292,741
Q_2C_3	WKHP ≤ 40 ∧ SCHL ≤ 19 ∧ PINCP_bucket IN (<20K, 20K-40K)	WKHP ≤ 38 ∧ SCHL ≤ 22 ∧ PINCP_bucket IN (<20K, 20K-40K, >60K)	≥125,492 SEX = F ∧ MAR = married; ≤9,628 SEX = M ∧ RAC1P = Asian	539,089→450,201
Q_3C_4	WKHP ≥ 60 ∧ SCHL ≤ 16 ∧ PINCP_bucket = <20K	WKHP ≥ 60 ∧ SCHL ≤ 16 ∧ PINCP_bucket IN (<20K, >60K)	≥1,307 SEX = F; ≥521 RAC1P = Black	4,367→13,177
Q_3C_5	WKHP ≥ 60 ∧ SCHL ≤ 16 ∧ PINCP_bucket = <20K	WKHP ≥ 60 ∧ SCHL ≤ 15 ∧ PINCP_bucket = <20K	≤2,829 RAC1P = White; ≤2,280 age_bucket = 30-60	4,367→1,553
Q_3C_6	WKHP ≥ 60 ∧ SCHL ≤ 16 ∧ PINCP_bucket = <20K	WKHP ≥ 66 ∧ SCHL ≤ 21 ∧ PINCP_bucket = <20K	≥521 RAC1P = Black; ≤2,280 age_bucket = 30-60	4,367→4,317
Q_4C_7	WKHP ≥ 60 ∧ SCHL ≥ 22 ∧ PINCP_bucket = >60K ∧ COW IN (self-emp-not-inc, self-emp-inc)	WKHP ≥ 60 ∧ SCHL ≥ 21 ∧ PINCP_bucket = >60K ∧ COW IN (self-emp-not-inc, self-emp-inc)	≥552 SEX = F; ≥97 RAC1P = Black; ≥239 MAR = divorced	2,343→4,997
Q_4C_8	WKHP ≥ 60 ∧ SCHL ≥ 22 ∧ PINCP_bucket = >60K ∧ COW IN (self-emp-not-inc, self-emp-inc)	WKHP ≥ 60 ∧ SCHL ≥ 23 ∧ PINCP_bucket = >60K ∧ COW IN (self-emp-not-inc, self-emp-inc)	≤1,862 RAC1P = White; ≤1,727 SEX = M	2,343→1,471
Tax [12, 45]. T1 from the earlier draft; T2, T3 and R5: Q' from our refinement search (R6 of the case-study search is the same pair as T2).				
T1	MaritalStatus = M ∧ Rate ≤ 5	Rate ≤ 5	≥30% HasChild = Y	57,164→114,957
T2	MaritalStatus = S ∧ HasChild = N ∧ Salary ≥ 50000	MaritalStatus = S ∧ Salary ≥ 50000	≥30% HasChild = Y	27,694→55,195
T3	Salary ≥ 60000 ∧ Rate ≤ 2	Salary ≥ 60000 ∧ Rate ≤ 3.9	≥40% Rate > 0	18,893→31,709
R5	Salary ≤ 30000 ∧ Rate ≤ 2	Salary ≤ 30000 ∧ Rate ≤ 3.2	≥50% Rate > 0	16,774→25,621

the same randomness, so it does not change the privacy analysis. Every other experiment uses the direct route of Section 4.2.

Metrics. All methods are scored on their output set with one yardstick, F_λ of Definition 3.7 with the unsmoothed *HD* ($\gamma_{\text{eval}} = 0$): smoothing is a device of the mechanism, and letting it define the metric too would make the comparison circular. The ApproxRatio is F_λ of the released set divided by F_λ of the baseline's set. A ratio says how good the released set is, not whether it is the same set, so we also report three set-overlap measures against the baseline's k explanations: *precision@k*, the share of the baseline's (view, attribute) pairs that the released set contains; the Jaccard index of the two sets; and *attribute precision*, the overlap of the two multisets of explained attributes, which gives credit for explaining the right attribute inside a different view.

5.4 Use Case Results

Each figure below shows explanations of one released run at the defaults ($\epsilon = 1, k = 5$), not chosen among runs. Bars are the released noisy histograms; the black ticks are the exact counts, drawn for the reader and never released. To keep the panels readable each shows the bins with the largest exact difference.

Adult: what the running example actually changes (A2). The refinement of Section 1 replaces `Bachelors` by `Masters` and lowers the hours threshold from 50 to 30, and the female share rises from 19.5% to 30.6%. Reading the two queries, one would credit the education predicate. The release points to the hours threshold instead (Figure 4). Among white holders of a master's degree, Q has no one in the 32–40 hour band and Q' has 999; the released counts, 61 and 985, leave no doubt about it. The same population moves in occupation and marital status: professional-specialty workers more than triple while executives only double, and the never-married grow from 165 to 584. This is the change the introduction describes: a population of long-hour, managerial, mostly married workers is replaced by a regular-hours, professional and more often unmarried one, and the female share rises as a consequence.

ACSIncome: a refinement from Erica (Q_1C_1). Erica satisfies the constraints on married women and Black workers by adding private-sector employees to a query over full-time government employees with some college education. The output grows from 141,100 to

Table 3: The query pairs of the experiments (continued).

ID	Q_1	Q_2	Constraint	$\lvert Q_1\rvert \to \lvert Q_2\rvert$
Tax-dirty [45], CQA. Σ = the Tax rules of [54] without `AreaCode` $\to$ `State`; *Q = US Census divisions picked by our selection rule.*				
C1	`State IN (AK, CA, HI, OR, WA)`	Q_Σ	–	18,729→16,404
C2	`State IN (AL, KY, MS, TN)` $\wedge$ `HasChild = N`	Q_Σ	–	7,986→7,082
C3	`State IN (AK, CA, HI, OR, WA)` $\wedge$ `Rate` ≤ 3	Q_Σ	–	8,126→6,152
Hospital [56], CQA. Σ = {`HospitalName` $\to$ `ZipCode`, `HospitalName` $\to$ `PhoneNumber`, `ZipCode` $\to$ `City`*}; H1 from the earlier draft, H2 and H3 by our selection rule.*				
H1	`HospitalType = acute care hospitals`	Q_Σ	–	968→93
H2	`EmergencyService = yes`	Q_Σ	–	830→91
H3	`HospitalOwner = government - hospital district or authority`	Q_Σ	–	198→54
IPUMS-CPS [63], contrasts. The questions of DPXPlain (I1 is its default question); the attribute that defines the groups is excluded from the views and the explanation attributes.				
I1	`SEX = Male`	`SEX = Female`	–	556,230 / 590,322
I2	`RELATE = Spouse`	`RELATE = Head/householder`	–	263,032 / 600,801
I3	`RELATE = Spouse`	`RELATE = Unmarried partner`	–	263,032 / 31,495
I4	`RELATE = Head/householder`	`RELATE = Child`	–	600,801 / 124,068
I5	`RELATE = Head/householder`	`RELATE = Parent`	–	600,801 / 23,629
I6	`MARST = Married, spouse present`	`MARST = Never married/single`	–	615,671 / 302,721
I7	`MARST = Divorced`	`MARST = Never married/single`	–	121,129 / 302,721
I8	`RACE = White`	`RACE = Black`	–	899,281 / 136,119
I9	`RACE = Asian only`	`RACE = White`	–	67,414 / 899,281
I10	`CITIZEN = Born in U.S`	`CITIZEN = Not a citizen`	–	949,180 / 89,876
I11	`CITIZEN = Naturalized citizen`	`CITIZEN = Born in U.S`	–	88,951 / 949,180
I12	`CLASSWKR = Wage/salary, private`	`CLASSWKR = Self-employed, not incorporated`	–	608,462 / 50,383
I13	`CLASSWKR = Federal government employee`	`CLASSWKR = State government employee`	–	23,005 / 40,551
I14	`AGE = (40, 50]`	`AGE = (30, 40]`	–	214,313 / 213,553
I15	`AGE = (60, 70]`	`AGE = (20, 30]`	–	146,860 / 181,255
I16	`EDUC = Bachelor's degree`	`EDUC = High school diploma or equivalent`	–	221,181 / 327,032
I17	`EDUC = Master's degree`	`EDUC = Bachelor's degree`	–	90,627 / 221,181

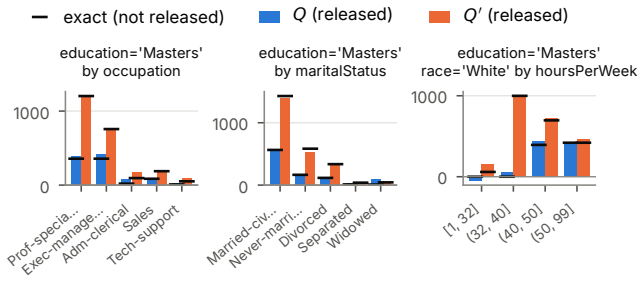


Figure 4: A2 (Adult, the running example): three of the five released explanations.

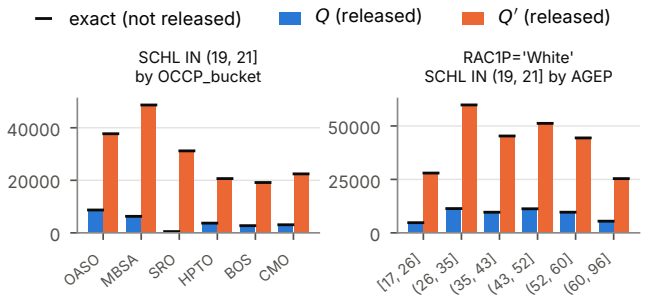


Figure 5: Q_1C_1 (ACSIncome, from Erica): two released explanations. `SCHL IN (19, 21]` is an associate's or bachelor's degree; SRO is sales, OASO office support, MBSA management.

587,712, which is expected; what the queries and the constraints do not say is what kind of workers arrive. Among holders of an associate's or bachelor's degree the release shows the occupational mix turning over (Figure 5): sales workers grow from 482 to 31,200, a factor of 65, against a factor of 4.3 for office support and 7.7 for management. A query about public-sector workers has become, for this education group, largely a query about salespeople, and an analyst comparing average incomes across the two outputs should know that before attributing the difference to the sector. At this size the noise, about 30 tuples per bin, is invisible.

Tax-dirty: the consistent answer is not a random sample (C1). Over the five Pacific states, `AVG(Rate)` is 4.74 on $Q(D)$ and 5.23 on $Q_\Sigma(D)$, although the consistent answer only drops 12% of the tuples. Two released explanations account for the whole difference (Figure 6). The first two panels show that the dropped tuples are not spread over the states: Alaska loses about half of its taxpayers, the other four states at most 7%. The third shows where the dropped tuples sat in the rate distribution: the lowest rate bucket shrinks by 22%, the top one by 2.5%. The two facts connect, because Alaska has no state income tax and every Alaskan tuple has rate 0. The repair semantics, not the taxpayers, moved the average: half of Alaska's tuples take part in some violation of the dependencies, and the consistent answer removes, disproportionately, the state with the lowest rates. An analyst who sees only the two averages has no way to know this.

IPUMS-CPS: where the income gap lives (I1). DPXPlain's default question asks why the average income of men is higher than that of women. The release finds the gap among married white respondents (Figure 7): in the top income bucket there are 84,299 men and 32,294 women, in the bottom bucket 11,638 men and 50,356 women. A second explanation gives one reason: among respondents outside

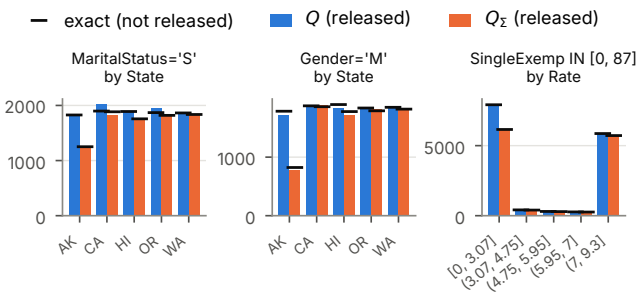


Figure 6: C1 (Tax-dirty, Pacific division, Q against its consistent answer Q_Σ): three released explanations.

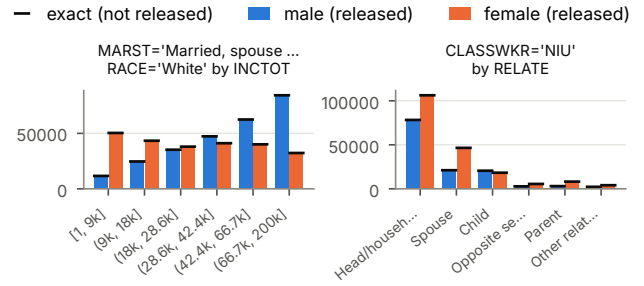


Figure 7: I1 (IPUMS-CPS, men against women): two released explanations. `CLASSWKR=NIU`: not in the labour force.

the labour force, more than twice as many women as men are the householder's spouse (46,532 against 21,129). DPXPlain answers the same question with a ranked list of predicates for the aggregate; the histograms here show the shape of the difference rather than its size alone.

When the data is too small: Hospital. The Hospital pairs show the other side. In H1 the consistent answer keeps 93 of 968 tuples: nearly every hospital takes part in some conflict, so there is little structure to find, and the file is itself almost all Alabama (923 of the 968 tuples), which is the view the algorithm returns. With $N = 1{,}000$ the bins hold a handful of tuples, and Laplace noise of scale $2k/\epsilon_H = 30$ per bin swamps most of them; only the largest differences, such as Birmingham dropping from 73 to 5 hospitals, survive the release. Section 5.5 shows that the view stage suffers too.

5.5 Quality

Figure 10 and Table 4 summarise the 44 pairs. At the default budget the released set reaches 0.94 and 0.96 of the optimum on Adult and Tax, 0.91 on IPUMS-CPS and 0.80 on ACSIncome; the two CQA datasets are the hard cases, at 0.63–0.64. B2 and B3 return sets of exactly the same score on every pair and at every ϵ, so non-private greedy is optimal on the τ-filtered candidates, as Proposition 7.1 predicts; we report them as one line. Several findings deserve comment.

Table 4: Quality at the defaults ($\epsilon = 1$), averaged over the pairs of each dataset, against B2/B3 unless marked.

Dataset	Ratio vs B1	Ratio	Attr. prec.	Prec.@k	Jaccard
Adult	1.02	0.94	0.80	0.40	0.29
ACSIncome	0.80	0.80	0.70	0.49	0.36
Tax	0.96	0.96	0.82	0.54	0.41
Tax-dirty	0.65	0.64	0.57	0.15	0.09
Hospital	0.73	0.63	0.57	0.15	0.09
IPUMS-CPS	0.91	0.91	0.87	0.34	0.23

A good set is not the same set. The ApproxRatio and the overlap measures tell different stories. On Adult the released set scores 0.94 of the optimum but contains on average only 40% of the optimum's explanations; on IPUMS-CPS 0.91 against 34%. The reason is that the leading candidates are nearly tied. A view and its refinements by one more predicate, such as `race = White` and `race = White` $\wedge$ `gender = Male`, explain the same attribute with almost the same histogram distance, and the exponential mechanism has no reason to prefer one of them; on Adult the relative gap between the best and second-best candidate is under 2% for a typical pair. Attribute precision, which forgives the choice of view, stays at 0.80–0.87 on these datasets: the release usually explains the right attributes, through a neighbouring view. For an analyst this is the relevant guarantee, but it has a practical consequence that ApproxRatio hides: two runs on the same pair return different, equally valid, explanation sets, and a user who compares runs should be told so. It also suggests that ApproxRatio alone, the measure used by earlier private explanation work, flatters any method whose candidate space contains many near-duplicates.

Noise sometimes helps. On Adult the ratio against B1 exceeds 1 at $\epsilon = 0.5$ and $\epsilon = 1$: the noisy algorithm finds better sets, by the paper's own unsmoothed measure, than the same algorithm without noise. The two objectives differ. B1 maximises HD_γ with $\gamma \approx 150$, and on Adult that γ exceeds the count of 79% of the non-empty bins, so the smoothed score favours candidates with large bins over candidates with sharp changes in small ones. Noise occasionally lands on the latter, and the unsmoothed yardstick rewards it. The effect is a measure of how far smoothing moves the target on a dataset with small bins, and it disappears on the large datasets, where B1 and the optimum coincide.

More data does not help the second stage. On ACSIncome and IPUMS-CPS the curves flatten early: from $\epsilon = 1$ to $\epsilon = 5$ the ratio moves from 0.80 to 0.84 and from 0.91 to 0.92, while Adult and Tax close their gap. The view stage is not the cause. Figure 11 measures it in isolation, as the overlap between the τ views it retains and the exact top-τ by *Use*: at the default ϵ_P the overlap is 0.94 or higher on every dataset except Hospital, and it is 0.98 on IPUMS-CPS even at $\epsilon_P = 0.03$. That is the sensitivity of *Use* at work, which shrinks as $1/N$. The second stage has no such help. The sensitivity of HD_γ is $1/(1+\gamma)$ whatever the size of the data (Proposition 3.9), and the distance it measures is relative, so a million tuples give the exponential mechanism no more separation between two candidates than fifty thousand do. What decides the second stage is the margin between the leading candidates, and

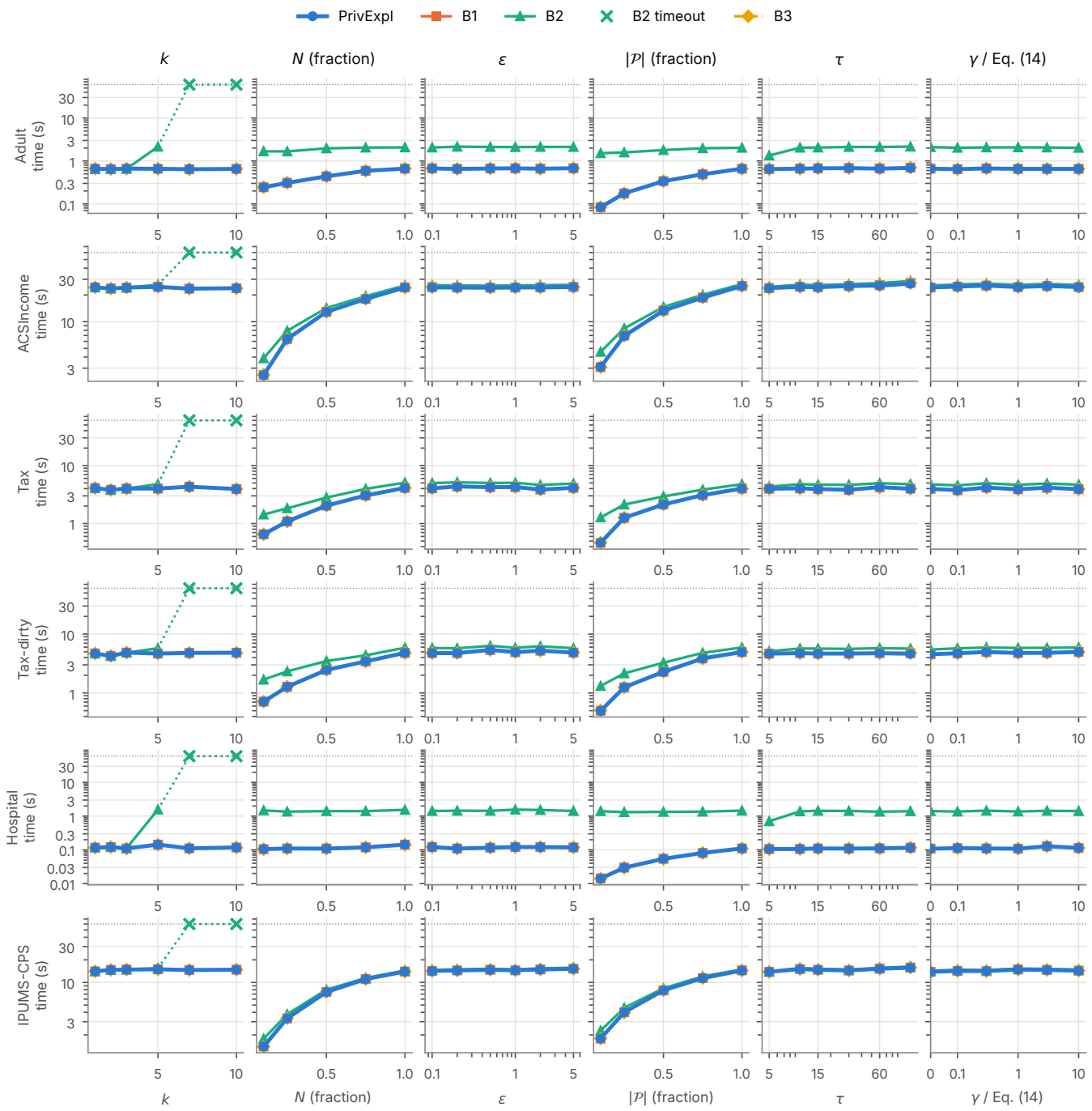


Figure 8: End-to-end running time, one row per dataset (mean over its runtime pairs), one column per varied parameter; all other parameters at the defaults. A cross marks a run stopped at the 60 s limit.

that is a property of the pair, not of N. The two stages therefore fail in different regimes: the first on small data, the second on pairs whose best explanations are close, at any size.

Where CQA is hard, and why. Hospital is the one dataset on which the view stage fails: at the default budget only 15% of the retained views belong to the exact top-τ, and $\epsilon_P = 10$ is needed to reach 0.89. With $N = 1{,}000$ the noise of the one-shot top-τ is large against the differences in *Use* between views, and the second stage then chooses among the wrong views. Tax-dirty has the opposite profile: the view stage is accurate (0.94) but the ratio is 0.64. Here

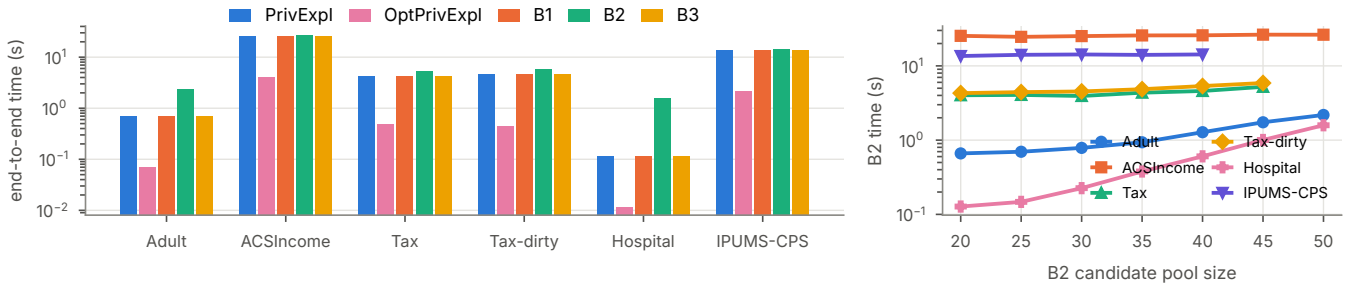


Figure 9: Left: end-to-end time of each method at the defaults (mean over the runtime pairs of each dataset). Right: B2 **end to end against the size of its candidate pool.**

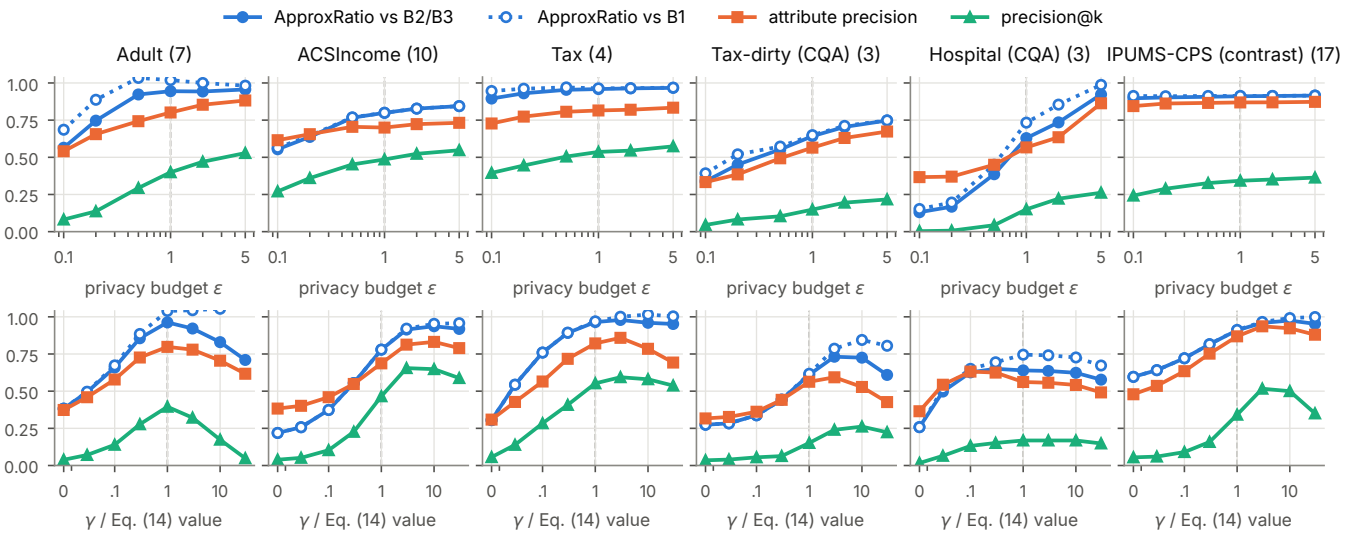


Figure 10: Quality of the released set against the baselines, averaged over the pairs of each dataset (number of pairs in parentheses), 40 **runs per point. Top: against the privacy budget. Bottom: against γ, as a multiple of the value of Eq.** (14) **at $\epsilon = 1$. Dashed vertical lines mark the defaults.** B2 **and** B3 **coincide.**

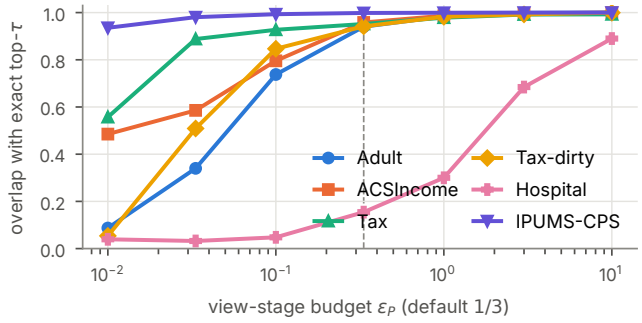


Figure 11: The view stage alone: overlap between the τ retained views and the exact top-τ by *Use*, **averaged over the pairs of each dataset (**40 **runs per point).**

the differences themselves are small. The conflicts are concentrated in a few states, so inside most views Q_Σ is Q minus a thin, even slice, and the best candidate of a typical pair has $HD_\gamma = 0.61$, less than the $2\Delta \ln |C_i|/\epsilon_i \approx 0.96$ within which the exponential mechanism guarantees to land; on the other datasets the best candidate scores 3.4 to 12.5, several times that bound. The second stage is then close to a uniform choice among reasonable candidates, which is what the ratio of 0.64 and the precision@k of 0.15 say. The difficulty comes from the size of the difference rather than from its CQA origin: Tax, a refinement dataset on the same schema, is the easiest of the six.

Budget split. We repeated the accuracy experiment with five splits of ϵ between the three stages. The ratio varies little: from 0.84 to 0.86 for every split that gives the view stage at least a tenth of the budget, with equal shares at 0.86; giving it 5% is the one clearly worse choice (0.80). Equal shares is therefore a safe default rather than a tuned one.

5.5.1 The γ Trade-off. The bottom row of Figure 10 varies γ from 0 to 30 times the value of Eq. (14). Without smoothing the algorithm

does not work at these budgets: averaged over all pairs the ratio is about 0.4 at $\gamma = 0$, against 0.86 at the default. Eq. (14) lands on the best value on Adult, where the ratio peaks at the default and falls on both sides, and on Hospital the curve is flat around it. On the datasets with large bins it is too small: the best value is three times the default on Tax and Tax-dirty and ten times on ACSIncome and IPUMS-CPS, and averaged over all pairs 3× gives 0.91 against 0.86. The reason is that Eq. (14) depends on τ, $|Attrs|$, k and ϵ_S but not on the data, while what γ competes with is the size of a bin. The median non-empty bin holds 20 tuples on Adult but 229 on IPUMS-CPS, 883 on ACSIncome and 898 on Tax, and the default $\gamma \approx 150$ exceeds the count of 79% of the bins on Adult but of only 5% on Tax. On Tax it is therefore almost no smoothing at all, and the exponential mechanism pays the full sensitivity without the benefit. A rule that also scales with a public proxy for the bin size, such as N/b, would fix the large datasets without hurting the small ones; we leave its analysis to future work. Precision@k shows the same optimum, so the gain is not an artefact of the ratio.

5.6 Scalability

Figure 8 varies one parameter at a time on every dataset. The private algorithm is linear in N and in $|\mathcal{P}|$ and flat in k, ϵ and γ, as the complexity analysis of Section 4.2 predicts: 97–98% of its time is the pass that computes *Use* for every view, which depends on neither k nor ϵ, and the k selection rounds take milliseconds. τ matters only through the explanation histograms of the retained views and adds 10% on ACSIncome at $\tau = 120$.

Privacy is free; exhaustiveness is not. B1, B3 and the private algorithm are within 0.5% of one another on every pair: they share the expensive pass, and the exponential mechanism replaces an arg max over the same scores. B2 is different in kind. At $k \leq 3$ it is as fast as the others, at $k = 5$ it adds a few seconds (from 0.7 s to 2.4 s on Adult, from 0.1 s to 1.6 s on Hospital), and at $k = 7$ it hits the time limit on every dataset, because its pool grows from 50 to 70 candidates and the number of subsets from 2.1×10^6 to 1.2×10^9. Since B3 returns the same sets as B2 wherever both finish, the exhaustive search buys nothing; the case for greedy selection is not only that it is fast but that, by Proposition 7.1, it loses nothing.

The cost is the statistics, and it can be cut. Since the private algorithm spends almost all its time computing statistics, that is where an optimisation pays. OptPrivExpl computes the same statistics in grouped passes and is 5.7–11.7× faster end to end (median 8.9×; Figure 9), from 25.8 s to 4.0 s on ACSIncome and from 14.0 s to 2.2 s on IPUMS-CPS. It released exactly the same output as the private algorithm in all 95 runs we compared (19 pairs, 5 runs each, same randomness), as it must, since it only changes how exact counts are computed. Its cost has a different shape, which Figure 8 shows: it is nearly flat in $|\mathcal{P}|$, because it pays per set of constrained attributes rather than per view, and it grows with τ, because the explanation histograms are now a visible part of its cost. At 10% of the views it is no faster than the direct route on ACSIncome, since there are few views per attribute set to share a pass; the gain comes with the realistic case of many views.

5.7 Summary of Findings

The released explanations exposed, in every setting, a change that neither the queries nor the constraint states: a refinement on Adult that works through the hours threshold rather than education, an Erica refinement that turns a public-sector query into one dominated by salespeople, a consistent answer whose average tax rate rises because the repair removes the one state without income tax, and an income gap concentrated among married respondents. Against the non-private optimum the released sets reach 0.80–0.96 of its score on the four non-CQA datasets at $\epsilon = 1$, while containing only a third to a half of its explanations, because the best candidates are near-ties; attribute precision shows that the right attributes are explained nonetheless. The view stage scales with N and is accurate everywhere but on the 1,000-tuple Hospital file; the explanation stage does not scale with N and is limited by the margin between candidates, which is why the two CQA datasets, whose differences are small or whose data is small, are the hard cases. The γ of Eq. (14) is near-optimal on data with small bins and too small, by a factor of three to ten, on data with large bins. Privacy costs nothing in running time, greedy selection costs nothing in quality, and the exhaustive alternative stops being feasible at $k = 7$; computing the statistics in grouped passes makes the whole pipeline about nine times faster with identical output.

6 RELATED WORK

We discuss three lines of work: how the pair of query outputs that we explain is produced, how existing systems explain query results and differences between data, and how differential privacy has been used for explanations. To our knowledge, no prior work generates differentially private explanations for the differences between two related query outputs.

Query refinement and repair. Query refinement has been studied for queries with empty answers [40, 52] and for queries that must meet constraints on the size of the output [20, 49]. Recent work refines queries to satisfy diversity and representation constraints. Erica [41, 42] changes a query so that the result meets constraints on the sizes of several subgroups. It uses provenance annotations to do this efficiently. The work in [60] finds a fair query that is close to the user's query. It supports range predicates and a binary sensitive attribute. The work in [15] changes the selection conditions of top-k queries. Provenance has also been used to analyze and reduce bias in data-driven tools [50]. In consistent query answering, queries are evaluated over the repairs of an inconsistent database [4, 10, 11]. These systems return a refined query or a repaired answer. They give limited support for explaining what the change means for the data. Our framework treats the process that produces the second output as a black box, so it can be used together with these systems (Section 2.2).

Explaining query results and data. Provenance [14, 18, 37] tracks why and where output data comes from. Intervention-based work [58] explains an aggregate result with predicates that select tuples whose removal changes the result. Scorpion [67] finds predicates that explain outliers in aggregate results using properties of the input tuples. Why-not questions [17, 64] ask why some tuples are missing from a query result. CAPE [48] explains an outlier in an aggregate result by values that deviate in the opposite direction.

MacroBase [6] explains outliers with attribute-value combinations that are common among outliers and rare among inliers. It ranks them with the relative risk ratio. DIFF [2] is a relational operator that provides the functionality of such explanation engines. Some works are closer to ours. SeeDB [65] scores a view by the distance between its distributions on a target subset and on a reference subset. FEDEX [22] scores a data exploration step by how much it changes the distribution of an attribute. Contrast set mining [8] and emerging patterns [24] look for conjunctions of attribute values whose distribution or support differs between groups or datasets. We use the interestingness, coverage, representativeness and diversity measures from this literature (Section 2.4). However, these works mainly explain a single query result, a prediction, or a data transformation. They do not explain the differences between two related query outputs. In addition, explanations based on individual tuples cannot be used under differential privacy [63].

Differential privacy for explanations. DPXPlain [63] explains aggregate query results under differential privacy. The user compares the aggregate values of two groups in a query result. DPXPlain releases noisy answers, a confidence interval for the difference between the groups, and a table of top-k explanation predicates. The predicates are selected with the one-shot top-k mechanism [26]. DPClustX [34] explains clustering results under differential privacy using histograms. It works in two stages. It replaces quality measures with high sensitivity by measures with low sensitivity. Like both works, we use private selection. Like DPClustX, we release noisy histograms and use quality measures with low sensitivity. Our setting is different. We explain the difference between two related query outputs, such as a query and its refinement. Our quality measures are defined for this comparison (Section 3.2). Each explanation is a pair of noisy histograms of an attribute inside a selected view. Systems for differentially private data analysis [38, 39, 47] support aggregate queries. As noted in [63], private SQL systems do not support explanations. We build our algorithms from known mechanisms: the exponential mechanism [46], one-shot top-k [26] and the Laplace mechanism [28]. We smooth the denominators of ratio-based scores to bound their sensitivity. A similar idea is used in differentially private decision trees [31] and in [34].

REFERENCES

[1] Serge Abiteboul, Richard Hull, and Victor Vianu. 1995. *Foundations of Databases*. Addison-Wesley. http://webdam.inria.fr/Alice/

[2] Firas Abuzaid, Peter Kraft, Sahaana Suri, Edward Gan, Eric Xu, Atul Shenoy, Asvin Ananthanarayan, John Sheu, Erik Meijer, Xi Wu, Jeff Naughton, Peter Bailis, and Matei Zaharia. 2018. DIFF: A Relational Interface for Large-Scale Data Explanation. *Proceedings of the VLDB Endowment* 12, 4 (2018), 419–432.

[3] Rakesh Agrawal, Tomasz Imieliński, and Arun N. Swami. 1993. Mining Association Rules Between Sets of Items in Large Databases. In *ACM SIGMOD International Conference on Management of Data*. 207–216.

[4] Marcelo Arenas, Leopoldo Bertossi, and Jan Chomicki. 1999. Consistent Query Answers in Inconsistent Databases. In *PODS*. 68–79.

[5] Abolfazl Asudeh, Zhongjun Jin, and H. V. Jagadish. 2019. Assessing and Remedying Coverage for a Given Dataset. In *IEEE International Conference on Data Engineering (ICDE)*. 554–565.

[6] Peter Bailis, Edward Gan, Samuel Madden, Deepak Narayanan, Kexin Rong, and Sahaana Suri. 2017. MacroBase: Prioritizing Attention in Fast Data. In *ACM SIGMOD International Conference on Management of Data*. 541–556.

[7] Solon Barocas, Moritz Hardt, and Arvind Narayanan. 2023. *Fairness and Machine Learning: Limitations and Opportunities*. MIT Press.

[8] Stephen D. Bay and Michael J. Pazzani. 2001. Detecting Group Differences: Mining Contrast Sets. *Data Mining and Knowledge Discovery* 5 (2001), 213–246. https://doi.org/10.1023/A:1011429418057

[9] Barry Becker and Ronny Kohavi. 1996. Adult. UCI Machine Learning Repository. https://doi.org/10.24432/C5XW20

[10] Leopoldo Bertossi. 2006. Consistent Query Answering in Databases. *SIGMOD Record* 35, 2 (2006), 68–76. https://doi.org/10.1145/1154448.1154461

[11] Leopoldo Bertossi. 2011. *Database Repairing and Consistent Query Answering*. Morgan and Claypool Publishers.

[12] Philip Bohannon, Wenfei Fan, Floris Geerts, Xibei Jia, and Anastasios Kementsietsidis. 2007. Conditional Functional Dependencies for Data Cleaning. In *Proceedings of the 23rd International Conference on Data Engineering (ICDE)*. 746–755.

[13] Mark Bun and Thomas Steinke. 2016. Concentrated Differential Privacy: Simplifications, Extensions, and Lower Bounds. In *Theory of Cryptography Conference*. Springer, 635–658.

[14] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. 2001. Why and Where: A Characterization of Data Provenance. In *ICDT*. 316–330.

[15] Felix S. Campbell, Alon Silberstein, Julia Stoyanovich, and Yuval Moskovitch. 2024. Query Refinement for Diverse Top-k Selection. *Proc. ACM Manag. Data* 2, 3, Article 166 (May 2024), 27 pages. https://doi.org/10.1145/3654969

[16] Jaime G. Carbonell and Jade Goldstein. 1998. The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries. In *ACM SIGIR Conference on Research and Development in Information Retrieval*. 335–336.

[17] Adriane Chapman and H. V. Jagadish. 2009. Why Not?. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data*. 523–534. https://doi.org/10.1145/1559845.1559901

[18] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. 2009. Provenance in Databases: Why, How, and Where. *Foundations and Trends in Databases* 1, 4 (2009), 379–474.

[19] Jan Chomicki, Jerzy Marcinkowski, and Slawomir Staworko. 2004. Computing Consistent Query Answers Using Conflict Hypergraphs. In *Proceedings of the 13th ACM International Conference on Information and Knowledge Management (CIKM)*. 417–426.

[20] Wesley W. Chu and Qiming Chen. 1994. A structured approach for cooperative query answering. *IEEE Transactions on Knowledge and Data Engineering* 6, 5 (1994), 738–749.

[21] Roni Copul, Nave Frost, Tova Milo, and Kathy Razmadze. 2024. TabEE: Tabular Embeddings Explanations. *Proc. ACM Manag. Data* 2, 1, Article 72 (March 2024), 26 pages. https://doi.org/10.1145/3639329

[22] Daniel Deutch, Amir Gilad, Tova Milo, Amit Mualem, and Amit Somech. 2022. FEDEX: An Explainability Framework for Data Exploration Steps. *Proc. VLDB Endow.* 15, 13 (Sept. 2022), 3854–3868. https://doi.org/10.14778/3565838.3565841

[23] Frances Ding, Moritz Hardt, John Miller, and Ludwig Schmidt. 2021. Retiring Adult: New Datasets for Fair Machine Learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[24] Guozhu Dong and Jinyan Li. 1999. Efficient Mining of Emerging Patterns: Discovering Trends and Differences. In *Proceedings of the Fifth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 43–52. https://doi.org/10.1145/312129.312191

[25] Marina Drosou and Evaggelia Pitoura. 2010. Search Result Diversification. *ACM SIGMOD Record* 39, 1 (2010), 41–47.

[26] David Durfee and Ryan M. Rogers. 2019. Practical Differentially Private Top-k Selection with Pay-what-you-get Composition. In *Advances in Neural Information Processing Systems (NeurIPS)*. 3527–3537.

[27] Cynthia Dwork and Jing Lei. 2009. Differential Privacy and Robust Statistics. In *Proceedings of the 41st Annual ACM Symposium on Theory of Computing (STOC)*. 371–380.

[28] Cynthia Dwork, Frank McSherry, Kobbi Nissim, and Adam Smith. 2006. Calibrating Noise to Sensitivity in Private Data Analysis. In *Theory of Cryptography*, Shai Halevi and Tal Rabin (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 265–284.

[29] Cynthia Dwork and Aaron Roth. 2014. The Algorithmic Foundations of Differential Privacy. *Found. Trends Theor. Comput. Sci.* 9, 3–4 (Aug. 2014), 211–407. https://doi.org/10.1561/0400000042

[30] Uriel Feige. 1998. A Threshold of $\ln n$ for Approximating Set Cover. *Journal of the ACM* 45, 4 (1998), 634–652.

[31] Arik Friedman and Assaf Schuster. 2010. Data Mining with Differential Privacy. In *Proceedings of the 16th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 493–502.

[32] Kareem El Gebaly, Parag Agrawal, Lukasz Golab, Flip Korn, and Divesh Srivastava. 2014. Interpretable and Informative Explanations of Outcomes. *Proceedings of the VLDB Endowment* 8, 1 (2014), 61–72.

[33] Liqiang Geng and Howard J. Hamilton. 2006. Interestingness Measures for Data Mining: A Survey. *Comput. Surveys* 38, 3 (2006), 9.

[34] Amir Gilad, Tova Milo, Kathy Razmadze, and Ron Zadicario. 2025. Differentially Private Explanations for Clusters. *Proc. ACM Manag. Data* (2025). Also available as arXiv:2506.05900.

[35] Sreenivas Gollapudi and Aneesh Sharma. 2009. An Axiomatic Approach for Result Diversification. In *International Conference on World Wide Web (WWW)*. 381–390.

[36] Francisco Herrera, Cristóbal José Carmona, Pedro González, and María José del Jesus. 2011. An Overview on Subgroup Discovery: Foundations and Applications. *Knowledge and Information Systems* 29, 3 (2011), 495–525.

[37] Melanie Herschel, Ralph Diestelkämper, and Houssem Ben Lahmar. 2017. A Survey on Provenance: What for? What Form? What From? *VLDB Journal* 26, 6 (2017), 881–906.

[38] Noah Johnson, Joseph P. Near, Joseph M. Hellerstein, and Dawn Song. 2020. Chorus: a Programming Framework for Building Scalable Differential Privacy Mechanisms. In *IEEE European Symposium on Security and Privacy (EuroS&P)*. https://doi.org/10.1109/EuroSP48549.2020.00041

[39] Noah Johnson, Joseph P. Near, and Dawn Song. 2018. Towards Practical Differential Privacy for SQL Queries. *Proceedings of the VLDB Endowment* 11, 5 (2018), 526–539.

[40] Nick Koudas, Chen Li, Anthony K. H. Tung, and Rares Vernica. 2006. Relaxing Join and Selection Queries. In *VLDB*.

[41] Jinyang Li, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Query Refinement for Diversity Constraint Satisfaction. *Proc. VLDB Endow.* 17, 2 (Oct. 2023), 106–118. https://doi.org/10.14778/3626292.3626295

[42] Jinyang Li, Alon Silberstein, Yuval Moskovitch, Julia Stoyanovich, and H. V. Jagadish. 2023. Erica: Query Refinement for Diversity Constraint Satisfaction. *Proc. VLDB Endow.* 16, 12 (2023), 4070–4073. https://doi.org/10.14778/3611540.3611623

[43] Ninghui Li, Min Lyu, Dong Su, and Weining Yang. 2016. *Differential Privacy: From Theory to Practice.* Morgan & Claypool Publishers. https://doi.org/10.2200/S00735ED1V01Y201609SPT018

[44] Hui Lin and Jeff A. Bilmes. 2011. A Class of Submodular Functions for Document Summarization. In *Annual Meeting of the Association for Computational Linguistics (ACL).* 510–520.

[45] Mohammad Mahdavi, Ziawasch Abedjan, Raul Castro Fernandez, Samuel Madden, Mourad Ouzzani, Michael Stonebraker, and Nan Tang. 2019. Raha: A Configuration-Free Error Detection System. In *Proceedings of the 2019 International Conference on Management of Data (SIGMOD).* 865–882.

[46] Frank McSherry and Kunal Talwar. 2007. Mechanism Design via Differential Privacy. In *FOCS.* 94–103.

[47] Frank D. McSherry. 2009. Privacy Integrated Queries: An Extensible Platform for Privacy-Preserving Data Analysis. In *Proceedings of the 2009 ACM SIGMOD International Conference on Management of Data.* 19–30. https://doi.org/10.1145/1559845.1559850

[48] Zhengjie Miao, Qitian Zeng, Boris Glavic, and Sudeepa Roy. 2019. Going Beyond Provenance: Explaining Query Answers with Pattern-based Counterbalances. In *Proceedings of the 2019 ACM SIGMOD International Conference on Management of Data.* 485–502.

[49] Chaitanya Mishra and Nick Koudas. 2009. Interactive query refinement. In *EDBT 2009, 12th International Conference on Extending Database Technology, Saint Petersburg, Russia, March 24-26, 2009, Proceedings (ACM International Conference Proceeding Series)*, Martin L. Kersten, Boris Novikov, Jens Teubner, Vladimir Polutin, and Stefan Manegold (Eds.), Vol. 360. ACM, 862–873. https://doi.org/10.1145/1516360.1516459

[50] Yuval Moskovitch, Jinyang Li, and H. V. Jagadish. 2022. Bias analysis and mitigation in data-driven tools using provenance. In *Proceedings of the 14th International Workshop on the Theory and Practice of Provenance, TaPP 2022, Philadelphia, Pennsylvania, 17 June 2022*, Adriane Chapman, Daniel Deutch, and Tanu Malik (Eds.). ACM, 1:1–1:4. https://doi.org/10.1145/3530800.3534528

[51] Yuval Moskovitch, Jinyang Li, and H. V. Jagadish. 2023. Detection of Groups with Biased Representation in Ranking. *CoRR* abs/2301.00719 (2023). https://doi.org/10.48550/arXiv.2301.00719 arXiv:2301.00719

[52] Ion Muslea and Thomas J Lee. 2005. Online query relaxation via bayesian causal structures discovery. In *AAAI.* 831–836.

[53] George L. Nemhauser, Laurence A. Wolsey, and Marshall L. Fisher. 1978. An Analysis of Approximations for Maximizing Submodular Set Functions—I. *Mathematical Programming* 14, 1 (1978), 265–294.

[54] Wei Ni, Xiaoye Miao, Xiangyu Zhao, Yangyang Wu, Shuwei Liang, and Jianwei Yin. 2024. Automatic Data Repair: Are We Ready to Deploy? *Proc. VLDB Endow.* 17, 10 (2024).

[55] Kobbi Nissim, Sofya Raskhodnikova, and Adam Smith. 2007. Smooth Sensitivity and Sampling in Private Data Analysis. In *Proceedings of the 39th Annual ACM Symposium on Theory of Computing (STOC).* 75–84.

[56] Theodoros Rekatsinas, Xu Chu, Ihab F. Ilyas, and Christopher Ré. 2017. HoloClean: Holistic Data Repairs with Probabilistic Inference. *Proc. VLDB Endow.* 10, 11 (2017), 1190–1201.

[57] Marco Túlio Ribeiro, Sameer Singh, and Carlos Guestrin. 2018. Anchors: High-Precision Model-Agnostic Explanations. In *AAAI Conference on Artificial Intelligence.* 1527–1535.

[58] Sudeepa Roy and Dan Suciu. 2014. A Formal Approach to Finding Explanations for Database Queries. In *ACM SIGMOD International Conference on Management of Data.* 1579–1590.

[59] Sunita Sarawagi, Rakesh Agrawal, and Nimrod Megiddo. 1998. Discovery-Driven Exploration of OLAP Data Cubes. In *International Conference on Extending Database Technology (EDBT) (Lecture Notes in Computer Science)*, Vol. 1377. 168–182.

[60] Suraj Shetiya, Ian P. Swift, Abolfazl Asudeh, and Gautam Das. 2022. Fairness-Aware Range Queries for Selecting Unbiased Data. In *38th IEEE International Conference on Data Engineering, ICDE 2022.* IEEE, 1423–1436. https://doi.org/10.1109/ICDE53745.2022.00111

[61] Abraham Silberschatz and Alexander Tuzhilin. 1996. What Makes Patterns Interesting in Knowledge Discovery Systems. *IEEE Transactions on Knowledge and Data Engineering* 8, 6 (1996), 970–974.

[62] Pang-Ning Tan, Vipin Kumar, and Jaideep Srivastava. 2002. Selecting the Right Interestingness Measure for Association Patterns. In *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD).* 32–41.

[63] Yuchao Tao, Amir Gilad, Ashwin Machanavajjhala, and Sudeepa Roy. 2022. DPXPlain: Privately Explaining Aggregate Query Answers. *Proc. VLDB Endow.* 16, 1 (2022), 113–126. https://doi.org/10.14778/3561261.3561271

[64] Quoc Trung Tran and Chee-Yong Chan. 2010. How to conquer why-not questions. In *Proceedings of the 2010 ACM SIGMOD International Conference on Management of data.* 15–26.

[65] Manasi Vartak, Sajjadur Rahman, Samuel Madden, Aditya G. Parameswaran, and Neoklis Polyzotis. 2015. SeeDB: Efficient Data-Driven Visualization Recommendations to Support Visual Analytics. *Proceedings of the VLDB Endowment* 8, 13 (2015), 2182–2193.

[66] Stefan Wrobel. 1997. An Algorithm for Multi-relational Discovery of Subgroups. In *European Symposium on Principles of Data Mining and Knowledge Discovery (PKDD) (Lecture Notes in Computer Science)*, Vol. 1263. 78–87.

[67] Eugene Wu and Samuel Madden. 2013. Scorpion: Explaining Away Outliers in Aggregate Queries. In *Proceedings of the VLDB Endowment*, Vol. 6. 553–564.

7 EXACT GREEDY SELECTION

Proposition 7.1 (The rounds are exact greedy). *Fix a candidate set $\mathcal{E}' \subseteq \mathcal{E}$ and a smoothing constant γ, and let $w_j = \lambda^{j-1}$.*

(i) The score $HD_\gamma(p, A) \cdot \lambda^{Counts[A]}$ that line 4 of Algorithm 2 assigns a remaining candidate is exactly its marginal gain in F_w, so selecting the highest-scoring candidate is one step of greedy on F_w.

(ii) For any non-increasing w, selecting $E \subseteq \mathcal{E}'$ with $|E| = k$ maximising $F_w(E)$ is solvable in polynomial time, and greedy returns an optimal solution.

Proof. (i) In a given round all candidates carrying an attribute A share the factor $\lambda^{Counts[A]}$, so whenever a round selects from that group it selects the member of largest HD_γ; after c selections from A the members taken are therefore its c highest-scoring ones. Every remaining candidate of A thus scores no higher than any already taken, so adding it places it at rank $c+1$ of its group and leaves the ranks of the existing members unchanged. The group term of Eq. (8) gains exactly $w_{c+1}h = \lambda^c h$, which is the score of line 4, and no other group term changes.

(ii) By Definition 3.7, $F_w(E) = \sum_A g_A(E)$ where g_A depends only on the members of E whose attribute is A. There is therefore no interaction between groups, and a solution is determined by how many candidates $n_A \geq 0$ it takes from each group, subject to $\sum_A n_A = k$, together with which ones. For a fixed n_A, the best choice within group A is its n_A highest-scoring candidates, since w is non-increasing and the scores are ranked (rearrangement inequality). Hence the value of taking g candidates from group A is

$$f_A(g) = \sum_{j=1}^{g} w_j \, h^A_{(j)},$$

whose successive marginal gains are $f_A(g) - f_A(g-1) = w_g \, h^A_{(g)}$. These are non-increasing in g, because w_g is non-increasing by

assumption and $h^A_{(g)}$ is non-increasing by construction; so each f_A is concave.

Maximising a separable sum of concave functions subject to a cardinality constraint is exactly the problem solved by the greedy rule of repeatedly taking the largest remaining marginal gain, which is optimal for separable concave resource allocation. Since each marginal gain is computable in constant time after sorting each group, the total cost is $O(|\mathcal{E}'| \log |\mathcal{E}'|)$ for $|\mathcal{E}'|$ candidates. With (i), the k rounds of Algorithm 2 run exactly this procedure. □

Example 7.2 (Greedy and the optimum agree). Let $\lambda = \frac{1}{2}$ and $k = 3$, and let five candidates carry the scores 9 and 4 on one attribute and 7, 6 and 2 on another. The marginal gains available are 9 and $\frac{1}{2} \cdot 4 = 2$ in the first group, and 7, $\frac{1}{2} \cdot 6 = 3$ and $\frac{1}{4} \cdot 2 = 0.5$ in the second. Greedy takes 9, then 7, then 3, for a total of 19. These are the three largest gains in the list, so no set of three candidates can do better, and enumerating all ten subsets confirms that 19 is the optimum. Taking the two best scores of the first group instead, 9 and 4, together with 7, gives 18: the second copy of an attribute is discounted, which is what makes the greedy order follow the discounted gains rather than the raw scores.